全国大学生嵌入式芯片与系统设计竞赛　FPGA 创新设计赛道

基于 Yosys 的 PMUX 结构优化设计

作品设计报告

参赛厂商　中科亿海微

开发方向　多路选择器 PMUX 结构优化

2026 年 10 月

摘　要

本项目基于 Yosys v0.69 开源综合工具，面向 Cyclone IV 的四输入 LUT 映射流程，设计并实现了一套由结构优化 Pass 与优化版 ABC9 命令流组成的面积优化方案。方案以**功能等价**为前提，评估**算法运行效率**及映射后的 **LUT 深度**，降低最终映射的组合逻辑单元数量，范围包括四输入 LUT 及独立反相器。项目在 Windows 11 下开发与测试，构建环境为 MSYS2 UCRT64，使用 GCC/G++ 16.2.0 编译器，通过 CMake 以 Release 模式构建。

Yosys/ABC 已具备成熟的通用逻辑优化能力，所以项目以特定结构的定向重组为核心路径，实现了成对选择结构共享、优先选择链重组、比较逻辑共享与使能条件利用、小规模查表逻辑分解与共享，以及完整译码选择网络区域重组五类优化规则。规则通过安全性检查与相应收益评估，对适用范围内的候选进行等价改写，实现对目标结构的通用优化的同时，有效防止对其他结构可能带来的负优化。ABC9 阶段分别生成同一 Pass 输出电路经逻辑重写后的映射候选与直接映射候选，通过面积优先的候选选择进一步减少 LUT。候选生成兼顾映射延迟，并对最终 LUT 深度进行评估。

在九个测试案例中，最终组合逻辑单元总数由 2427 降至 1985，逐例平均削减率为 **24.21%**，各例触发器数量均不增加，最大 LUT 层数五例不变、两例降低、两例增加。等价性验证使用 OSS CAD Suite 中的 EQY，验证覆盖原始 RTL 与基线网表、原始 RTL 与优化网表、基线与优化网表三组比较，九例共 **27 项验证全部通过**。工程采用统一的测试与统计脚本，通过预热及必要时的多轮复核，评估墙钟时间与主进程峰值内存占用。十轮测试中，所有案例时间开销均低于 5%，八例的时间与内存均十轮达标，另一例两项同时达标 5 轮。此外，自行开展额外变体测试和压力测试，用于验证功能等价性，并评估方案的通用性及适用边界。

关键词：PMUX；逻辑综合；LUT 映射；Yosys；ABC9；形式等价验证

目录

摘要 i

第一部分 作品概述 1

1.1 优化目标与约束 1
1.2 总体方案与综合流程 1
1.3 主要技术指标 2

第二部分 优化方法与实现 4

2.1 Pass 的接入位置与执行机制 4
2.2 P1 成对选择结构的共享 5
2.3 P2 优先选择链的重组 10
2.4 P3 比较逻辑共享与使能条件利用 13
2.5 P4 小规模查表逻辑分解与共享 17
2.6 P5 完整译码选择网络区域重组 21
2.7 ABC9 候选映射与面积优先选择 25

第三部分 正确性验证与性能评估 27

3.1 测试配置与评价方法 27
3.2 功能等价性验证 27
3.3 面积优化效果与贡献分析 29
3.4 运行时间与峰值内存评估 32
3.5 补充压力测试 34

第四部分 工程组织与复现 35

4.1 交付内容与文件组织 35
4.2 运行与重新构建 35

参考文献 36

第一部分 作品概述

1.1 优化目标与约束

项目针对 Yosys 综合流程中的多路选择及相关典型逻辑结构开展面积优化。目标是在保持电路功能等价的前提下，通过 RTL 结构重组与 ABC9 映射流程优化，减少 Cyclone IV 器件映射后的组合逻辑资源消耗。

面积指标采用最终映射的四输入 LUT 与独立反相器数量之和。单个测试案例的优化率定义为：

$$R_i = \frac{N_{\text{base},i} - N_{\text{opt},i}}{N_{\text{base},i}} \times 100\%$$

其中，$N_{\text{base},i}$ 与 $N_{\text{opt},i}$ 分别表示基线版本和优化版本的组合逻辑单元数量。总体效果按照要求采用逐例算术平均值进行评价。

方案设计遵循以下约束：

1. **功能等价。**不改变电路的输入输出行为、寄存器初态及复位语义；仅在满足结构条件和安全限制时实施改写。
2. **比较条件统一。**基线与优化版本采用相同的 Yosys、ABC 版本、编译工具链及构建配置，共用同一份外部综合脚本，在同一环境下进行测试。
3. **效率开销受控。**统计完整综合过程的时间与主进程峰值内存占用，以满足命题规定的效率要求。时间方面根据赛事方建议，使用 PowerShell 的 `Measure-Command` 统计，实际测得的是从启动 Yosys 到其退出的墙钟时间；内存通过 Windows API `GetProcessMemoryInfo` 读取 Yosys 主进程的峰值工作集。
4. **评估 LUT 深度。**同时统计映射后串联 LUT 的最大层数，结合面积收益分析逻辑深度变化，为不同路径延迟要求下的方案选择提供依据。

1.2 总体方案与综合流程

Yosys/ABC 的通用优化构成本项目的综合基础。面向四输入 LUT 进一步压缩面积，需要利用特定结构中可共享的数据、控制条件和中间函数，调整其逻辑组织方式。设计据此采用“结构优化 Pass ＋ ABC9 候选映射选择”的两阶段方案。

第一阶段在 RTLIL 网表上执行自定义 `pmux_opt` Pass。Pass 依据网表连接与函数关系识别 PMUX 及相关结构，按适用条件、安全限制及相应成本判断选择改写对象，减少重复选择和比较及不利于四输入 LUT 映射的逻辑组织形式。Pass 由五类规则组成：

表 1.1 结构优化规则及面向对象

规则	面向优化对象
P1 成对选择结构共享	两组具有成对交换关系的选择结构
P2 优先选择链重组	满足条件的多级优先选择链
P3 比较逻辑共享与使能条件利用	具有公共比较信息或条件约束的逻辑
P4 小规模查表逻辑分解与共享	可分解并共享中间函数的小规模查表结构
P5 完整译码选择网络区域重组	完整译码控制的树形及共享选择网络

第二阶段使用优化版 ABC9 命令流完成逻辑优化与 LUT 映射。该阶段以经过 Pass 优化的同一电路为输入，分别生成“逻辑重写后映射”与“直接映射”两种映射候选，通过面积优先的候选选择进一步减少 LUT。候选生成过程仍兼顾映射延迟，最终结合面积与最大 LUT 层数评价映射结果。

整体数据流如下：



五类规则统一接入同一综合位置，并使用固定的 ABC9 命令流处理不同电路。各规则分别定义适用结构及改写条件，满足条件的候选进入相应变换，其余部分继续原有综合流程。第二部分先说明结构关系与变换原理，再结合具体案例展示其应用及实现。

1.3 主要技术指标

在提供的九项测试案例中，设计取得的优化效果以及各项效率表现如下：

优化效果

表 1.2 九个测试案例的资源优化与等价性验证结果

案例	LUT	独立 NOT	LUT + NOT	优化率	DFF	LUT 深度	等价性验证
test1	286 → 182	2 → 2	288 → 184	36.11%	222 → 222	4 → 4	通过
test2	25 → 15	0 → 0	25 → 15	40.00%	0 → 0	2 → 2	通过
test3	12 → 7	1 → 1	13 → 8	38.46%	17 → 14	3 → 3	通过
test4	35 → 25	1 → 1	36 → 26	27.78%	10 → 10	3 → 2	通过
balanced	426 → 366	0 → 0	426 → 366	14.08%	12 → 12	10 → 9	通过
pipeline	450 → 348	0 → 0	450 → 348	22.67%	24 → 24	4 → 6	通过
crossbar	408 → 361	0 → 0	408 → 361	11.52%	10 → 10	6 → 7	通过
feedback	367 → 298	1 → 1	368 → 299	18.75%	16 → 16	7 → 7	通过
nested	413 → 378	0 → 0	413 → 378	8.47%	12 → 12	10 → 10	通过
资源合计	2422 → 1980	5 → 5	2427 → 1985	—	323 → 320	—	全部通过
逐例平均	—	—	—	24.21%	—	—	—

逐例优化率的算术平均为 24.21%，达到满分指标要求的 20%。

效率

表 1.3 九例十轮测试的效率开销范围与达标轮数

案例	时间开销 参考均值（%）	墙钟时间开销 十轮范围（%）	内存开销 参考均值（%）	峰值工作集开销 十轮范围（%）	两项均 ≤ 5% 轮数
test1	−11.69	−23.89 ～ −8.08	−3.94	−5.34 ～ −3.21	10/10
test2	−2.03	−11.95 ～ 2.36	+0.25	−1.34 ～ 1.36	10/10
test3	−0.35	−5.74 ～ 3.23	+0.07	−0.49 ～ 0.56	10/10
test4	−4.38	−12.62 ～ 2.47	−1.35	−2.58 ～ 0.21	10/10
balanced	−8.81	−18.21 ～ 2.61	−4.90	−7.65 ～ −2.96	10/10
pipeline	−13.75	−18.48 ～ −10.94	−1.26	−1.86 ～ −0.49	10/10
crossbar	−1.96	−6.38 ～ 2.52	+5.12	4.23 ～ 6.42	5/10
feedback	−3.53	−7.05 ～ 0.60	−1.94	−2.76 ～ −0.72	10/10
nested	−8.97	−16.66 ～ −3.82	−1.44	−2.79 ～ 0.84	10/10

九例的时间开销均低于 5%；八例在十轮中时间与内存均达标。crossbar 的内存开销均值为 5.12%，仅高于 5% 阈值 0.12 个百分点；十轮结果在 4.23%～6.42% 之间波动，其中 5 轮达标，呈现接近阈值的波动特征。测量、预热及复核方法见第 3.4 节。

第二部分 优化方法与实现

本部分围绕五类选择、比较与查表结构，说明其连接关系、等价变换及适用条件，再结合具体电路展示识别与改写过程。规则依据网表中的结构和函数关系匹配候选，并通过安全检查、规模限制及相应收益条件控制改写范围。下文先介绍 Pass 的接入与执行机制，再依次展开各类结构优化及配套 ABC9 映射方法。

2.1 Pass 的接入位置与执行机制

`pmux_opt` 随优化版 Yosys 编译并链接到主程序，运行时无需加载动态插件。两版测试共用 `project/runner/synthesis.ys`，预热、正式测量和复核均调用此文件，分别保存输出；优化差异来自程序内部。

Pass 直接操作 Yosys 的 RTLIL 中间表示，在模块、单元和信号连接层面完成结构分析与改写 [1]。

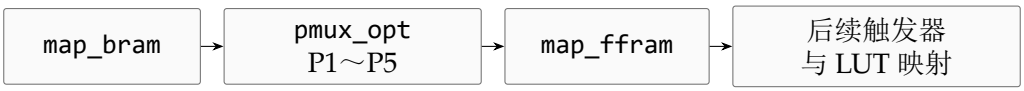
2.1.1 接入位置与选择依据

项目在 `techlibs/intel/synth_intel.cc` 中增加独立的 `pmux_opt` 阶段，放在 `map_bram` 之后、`map_ffram` 之前。此时，过程转换与通用优化已经完成，PMUX、比较逻辑及未映射的 ROM 等结构仍然保留，便于进行结构识别与共享分析。若过早执行，部分目标结构尚未形成；若在存储展开和基础门级映射后执行，原有宽位选择关系及 ROM 信息可能已被拆散。

先执行块 RAM 映射，可以让符合条件的存储结构优先使用专用资源，再由 P4 处理保留下来的小规模异步 ROM。新增阶段的关键调用为：

```
if (check_label("pmux_opt")) {
    run("pmux_opt -p1 -p2 -p3 -p4 -p5");
}
```

该阶段与前后处理的局部关系如下，完整综合流程见第 1.2 节。



2.1.2 命令入口与规则调度

`pmux_opt.cc` 中的 `PmuxOptPass` 继承自 Yosys 的 Pass 基类，其构造函数通过 `Pass("pmux_opt", ...)` 注册命令，并通过全局实例接入命令注册机制。`help()` 提供参数说明，`execute()` 负责参数解析、设计选择范围检查及规则调度。

命令未指定规则参数时，默认启用 P1～P5；通过 `-p1`、`-p2`、`-p3`、`-p4` 或 `-p5` 可仅启用所选规则，也可组合使用，`-all` 则启用全部规则。`-v` 用于输出详细的匹配与改

写信息。

`execute()` 按 P1、P2、P3、P4、P5 的固定顺序执行已启用规则，日志先列出启用规则及所选设计的 PMUX 数量，处理结束后汇总改写次数。P5 发生改写时，还会调用 `opt_clean` 清理无用单元。后续综合阶段继续完成逻辑展开及 LUT 映射。

2.2 P1 成对选择结构的共享

P1 面向选择信号相同、部分数据通道具有交叉交换关系的两个 PMUX，通过共享组内数据选择并统一恢复输出顺序，减少重复的选择逻辑。识别依据是网表连接与数据相等关系，下文以 test1 展示其应用。

2.2.1 优化对象与案例结构

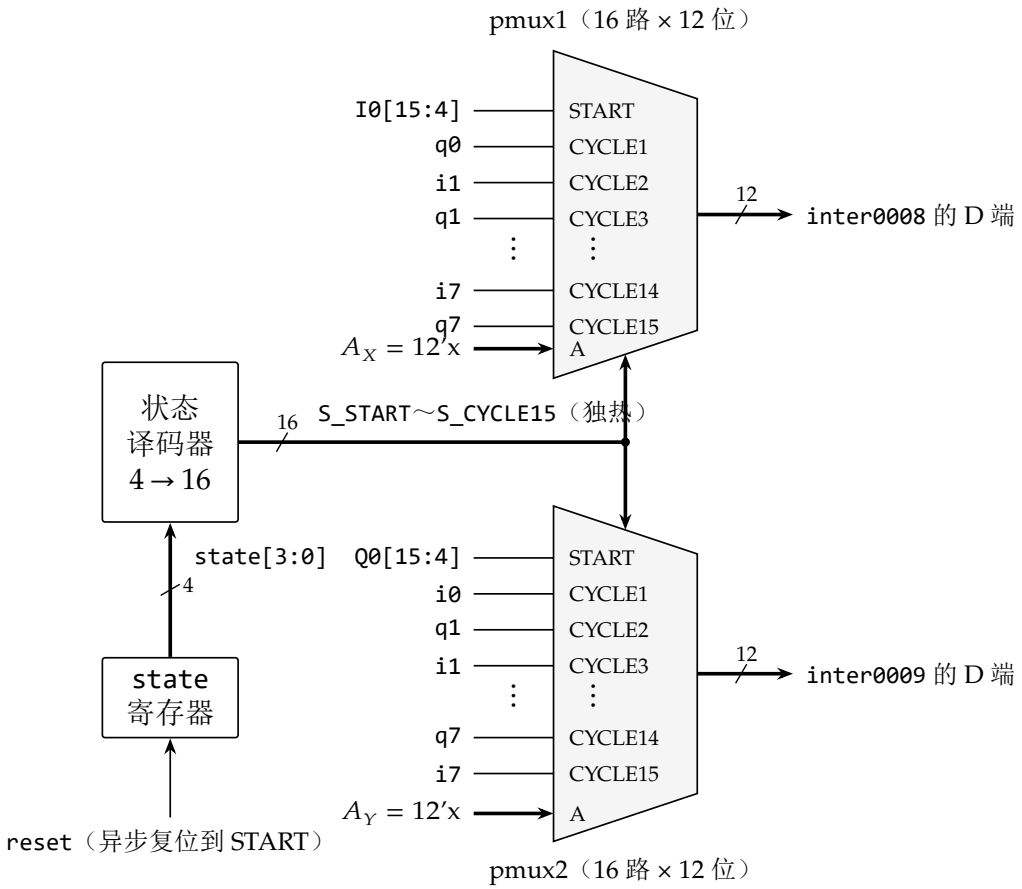


图 2.1 test1 改写前的选择结构（A 为默认输入）

如图 2.1 所示，test1 的主体结构是一个 16 状态的状态机。状态译码器先把 4 位的 `state[3:0]` 译为 16 位独热码 `S_START～S_CYCLE15`，每一位表示一个状态。两个 12 位寄存器 inter0008、inter0009 的 D 端各接一个 16 路的 `$pmux`，两个 `$pmux` 使用同一组独热码作为选择信号，在每个状态下各自从自己的 16 路输入中选出一组数据。reset 信号接在 state 寄存器上，复位时状态回到 START。

下表列出 `state[3:0]` 的每个取值下 inter0008 与 inter0009 装入的数据。表中按 `state[3:1]` 把 16 个状态分为 8 行，根据 `state[0]` 划分每行的两个状态：

表 2.1 test1 各状态下两个输出装入的数据

state[3:1]	state[0]=0			state[0]=1		
	状态	inter0008	inter0009	状态	inter0008	inter0009
0	START	I0[15:4]	Q0[15:4]	CYCLE1	q0	i0
1	CYCLE2	i1	q1	CYCLE3	q1	i1
2	CYCLE4	i2	q2	CYCLE5	q2	i2
……	……	……	……	……	……	……
7	CYCLE14	i7	q7	CYCLE15	q7	i7

I0、Q0 为 16 位输入端口，i0～i7、q0～q7 为 12 位寄存器，在 START 的采样沿保存对应输入的 [15:4] 位。CYCLE1 使用已保存的数据，输入端口则可继续变化，因此两者是不同的组合信号来源。

原版映射为两个输出分别实现数据选择；不同共享范围的资源对照见第 3.3.3 节。

2.2.2 优化逻辑的等价变换

表 2.1 的第 1～7 行具有相同规律：两个状态使用同一对数据 i_k, q_k，仅交换它们与两个输出的对应关系。因此可以先选择数据对，再用一个公共控制信号恢复两个输出的顺序。

第 0 行使用不同数据来源：START 直接使用输入端口，CYCLE1 使用 START 时保存到寄存器的数据，不满足组合网表上的直接交叉相等条件。P1 通过局部 MUX 处理这一组，再与后面七组共同接入共享 PMUX，称为“部分交换”。下面给出同时覆盖这两种情况的等价变换。

考虑两个输出 X、Y 和一对状态 s、t。状态 s 下 X 取 x_s、Y 取 y_s，状态 t 下 X 取 x_t、Y 取 y_t。用 t 同时表示状态 t 的单比特选通信号；在本组被选中时，$t = 0$ 对应状态 s，$t = 1$ 对应状态 t。为这一对状态定义两个中间值 L 与 H：

$$L = t\,?\,x_t : y_s,$$

$$H = t\,?\,y_t : x_s.$$

于是在这两个状态下都有：

$$X = t\,?\,L : H,$$

$$Y = t\,?\,H : L.$$

无论 x_s、y_s、x_t、y_t 取何值，这一变换都成立。而当两个输出的值在这对状态间存

在交换关系，即 $x_t = y_s$ 且 $y_t = x_s$ 时，L 恒为 x_t，H 恒为 y_t，定义 L、H 的两个 2 选 1 都退化为直接连线，不产生任何逻辑。

现在把这种变换推广到全部状态：把全部状态两两配成一对，第 k 对记为第 k 组，组内两个状态记为 s_k、t_k，它们在独热码中对应的选择位记为 $\mathrm{S}(s_k)$、$\mathrm{S}(t_k)$。在此基础上定义两类信号：

1. 组选择信号 $G_k = \mathrm{S}(s_k) \mid \mathrm{S}(t_k)$，表示当前状态属于第 k 组。原选择端至多一位为 1 时，组选择信号也至多一位为 1；在 test1 的有效状态下，G0～G7 中恰有一位有效。

2. 交换信号 $\mathrm{swap} = \mathrm{S(t0)} \mid \mathrm{S(t1)} \mid \cdots \mid \mathrm{S(t7)}$，表示当前状态是所在组中的 t。

每组的 $\{H_k, L_k\}$ 接入以 G_k 为选择端的共享 PMUX，得到 $\{H, L\}$；再以 swap 控制是否对调高低两半，恢复两个输出。在 P1 中，根据两个输出的值在这对状态间是否存在交换关系，各组的 L_k、H_k 有下面两种实现方式：

1. 对于存在交换关系的组（即 test1 的第 1～7 行）：L_k、H_k 的值即为 pmux 的第 k 路输入值，可以直接共享 `$pmux` 的第 k 路输入。

2. 对于不存在交换关系的组（即 test1 的第 0 行）：L_k、H_k 各需要一个以 $\mathrm{S}(t_k)$ 为选择端的 mux，这两个 mux 的输出拼起来作为共享 `$pmux` 的第 k 路输入。后文将这种处理方式称为部分交换。

经过等价变换改写之后 test1 中的电路如下图 2.2 所示。在 test1 中取 X 为 inter0008、Y 为 inter0009，每组的 t 为偶数状态（START、CYCLE2、……、CYCLE14），s 为奇数状态。L_k 是该行交给 inter0008 的 I 类数据，H_k 是 Q 类数据，下文分别记它们为 $ilane_k$、$qlane_k$；swap 为 8 个偶数状态选择位的或，记为 even。

test1 的两个原 PMUX 默认输入均为 12 位 X，如图 2.1 的 A 端所示；4 位状态的全部 16 个码值已被译码覆盖。实现保留两个原 PMUX 的 A 端，将共享结果在 $active = \bigvee_k G_k$ 有效时接入。test1 中全部通道均已共享，且 *active* 在确定状态下恒为 1，因此图 2.2 直接画出共享结果到输出的连接。共享 PMUX 的 A 端为 24 位 X；对于一般结构，$active = 0$ 时，输出仍由原 PMUX 选择各自的默认输入。

2.2.3 相关结构识别和改写

实现入口为 `run_group_rule()`，处理过程分为以下四步。

候选筛选。逐模块收集尚未处理的 PMUX，仅考虑输入路数为不小于 6 的偶数、数据宽度和输入路数相同、且选择端信号一致的两个单元。

交换关系计数。按 RTLIL 通道顺序将第 $2k$、$2k+1$ 路组成一组。若两个单元的数据交叉相等，且同一单元这两路数据不同，则计为交换组。对每个起始单元选择交换组数最多的配对对象，少于三组则不改写。该门槛用于过滤共享不足的候选。

确定共享范围。若全部组都满足交换关系，直接共享全部输入；否则先进行组合环

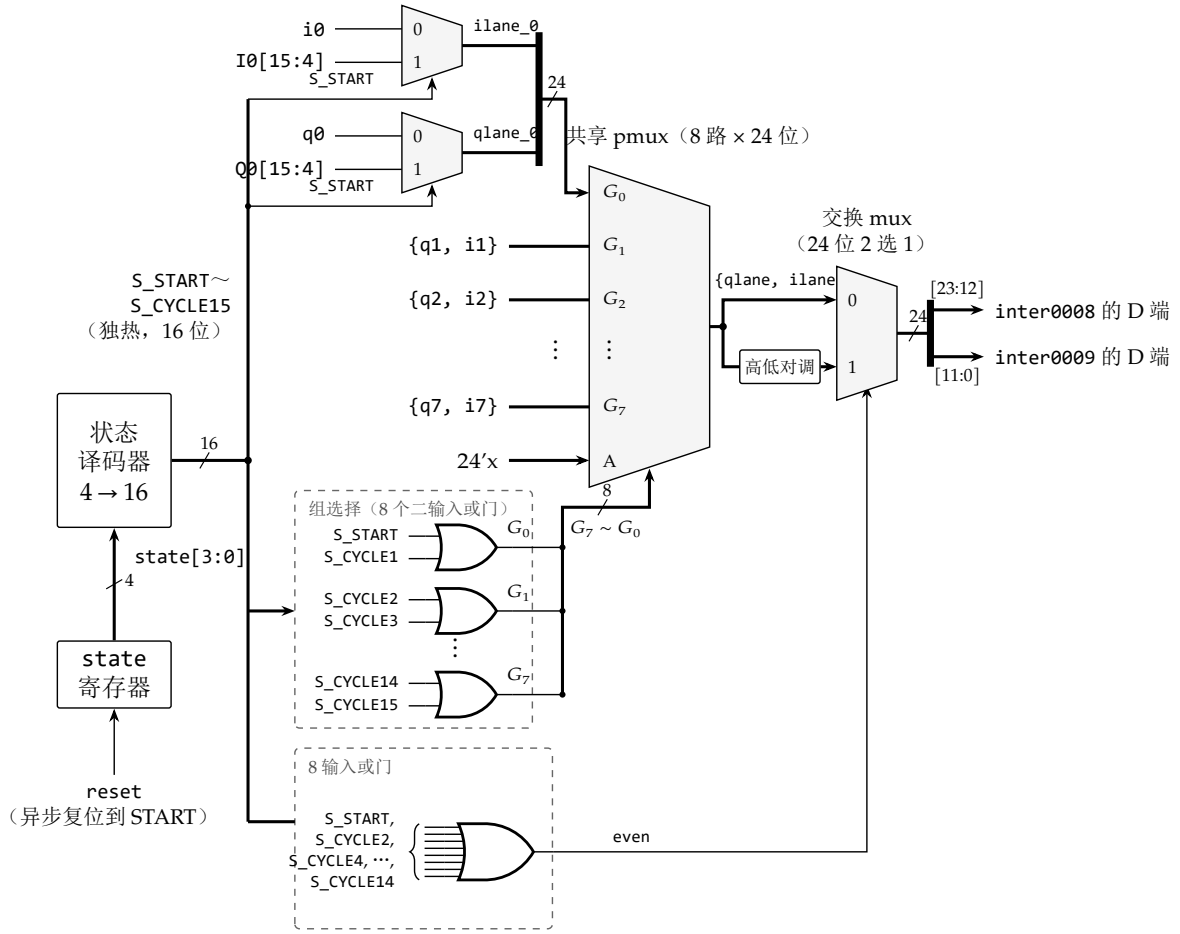


图 2.2　test1 改写后的选择结构（共享 PMUX 默认输入为 24 位 X）

检查。检查通过时，将非交换组按局部 MUX 方式一并纳入；未通过时，仅共享交换组，其余通道留在原 PMUX 中。

生成与替换。按图 2.2 建立组选择、共享数据对及统一交换网络，并保留两个原 PMUX 各自的默认输入。未共享的通道继续使用原选择位；共享部分由一个汇总使能接回原输出。配对后的两个单元标记为已处理，不再参与其他配对。核心生成过程如下，其中 $u = 2k, v = 2k + 1$：

代码 2.1　P1 共享结构生成伪代码

```
for each selected group k = (u, v):
    G[k] = S[u] | S[v]
    L[k] = pick(A[u], B[v], S[v])
    H[k] = pick(B[u], A[v], S[v])
raw = pmux(x, {H[k], L[k]}, G[k])
result = swap_halves_if(raw, OR(S[v]))
active = OR(G[k])
rebuild A, B: keep unselected lanes and original defaults;
              add the corresponding half of result under active
```

其中 pick 的两个数据相同时直接返回该信号，否则生成二选一 MUX。

在 test1 中，全部 8 组输入均进入共享网络：7 组直接共享，1 组增加局部 MUX。RTLIL 的通道顺序与状态数值顺序相反，因此伪代码中第 v 路选择位对应偶数状态，汇

总后即为图中的 even。这里的相邻配对是当前实现控制搜索成本的限制；规则并未穷举任意通道配对。

2.2.4 正确性与安全条件

输出行为保持 选择端全为 0 时，改写后的两个输出仍取各自原默认值。恰好一路有效时，已共享的通道先由共享 PMUX 选出数据对，再恢复输出顺序；未共享的通道继续由两个原 PMUX 选择。因此两种情况下的输出均与原结构相同。多路同时有效时，原 $pmux 的输出为未定义值 [2]；test1 的选择位来自不同状态的译码，确定状态下不会同时有效。

部分交换检查组合环 考虑下面的合法单向连接：第一个 PMUX 产生 X，第二个 PMUX 将 X 作为一路输入，再产生 Y。

单元	输入数据	输出
第一个 PMUX	A、B	X
第二个 PMUX	X、C	Y

原来是“先得到 X，再计算 Y”。若把两者全部合并进一个共享网络，新网络既要读取第二个 PMUX 的输入 X，又负责产生 X，就会出现下面的反馈：

$$\text{原连接：}\quad \mathrm{PMUX}_X \longrightarrow X \longrightarrow \mathrm{PMUX}_Y \longrightarrow Y,$$

$$\text{不安全的合并：}\quad X \longrightarrow \text{共享网络} \longrightarrow X.$$

也就是产生 X 的组合逻辑需要先知道当前 X，形成组合环。若反馈路径中隔着寄存器，读取的是寄存器保存的数据，不属于这种同一时刻的组合环。

完全交换组的两边本就使用同一对数据，改写不增加数据来源；部分交换则会把两边原有的不同数据汇入共享网络，因此必须额外检查。代码从两个待改写 PMUX 的数据、默认值及选择输入出发，沿组合驱动向上追踪：若找到任一待改写输出，或遇到不能可靠分析的单元，就拒绝部分交换；追踪到触发器时停止。

检查不通过时的改写 检查不通过时，规则只共享完全交换的组，其他通道和两个默认输入留在原 PMUX 中；共享结果作为新增通道接回两个瘦身后的原 PMUX。此时不生成非交换组的两个局部 MUX，统一交换 MUX 仍保留。

图 2.2 是 test1 检查通过、八组全部合并的结构。图 2.3 则示意第一组不能并入时的另一种结果：只共享后七组，前两个状态的通道留在两个原 PMUX 中。

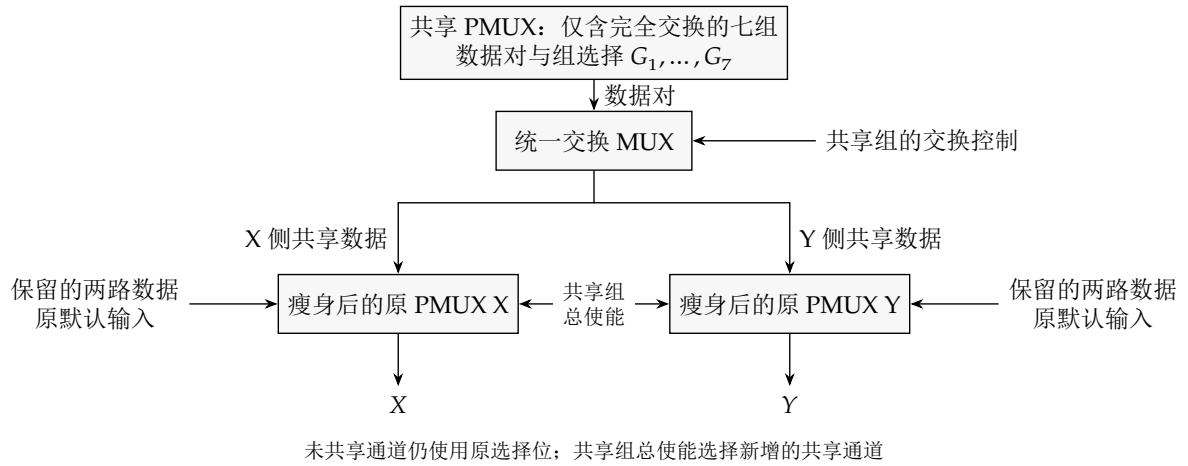


图 2.3　仅共享完全交换组、保留原 PMUX 残余通道的结构示意

可能成环时，P1 将共享范围缩小至完全交换组，保留原有周期行为及寄存器参数。整设计等价性验证见第 3.2 节。

2.3　P2　优先选择链的重组

规则 P2 针对综合过程中出现的四级优先 MUX 链，通过重组数据通路，改善后续四输入 LUT 映射的面积与 LUT 深度 [2]。

test2 的选择优先级为 $a > b > c > d$，未命中时输出零，综合后形成图 2.4 的四级 MUX 链。

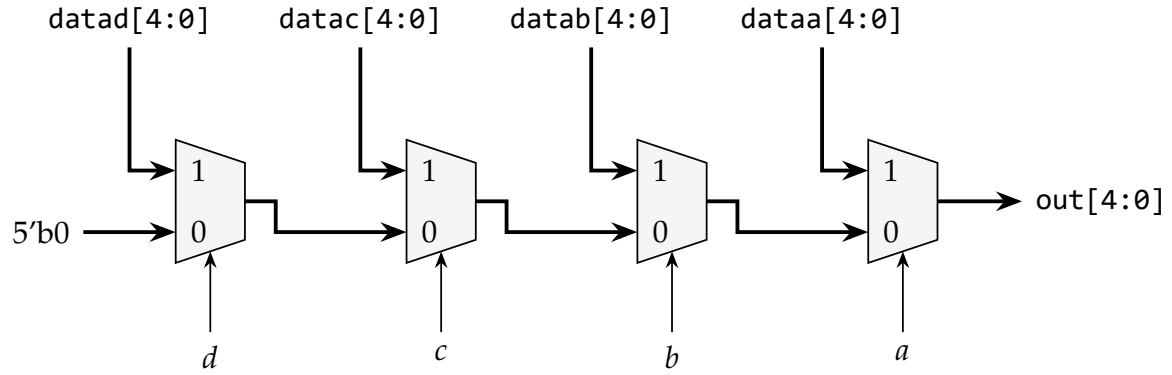
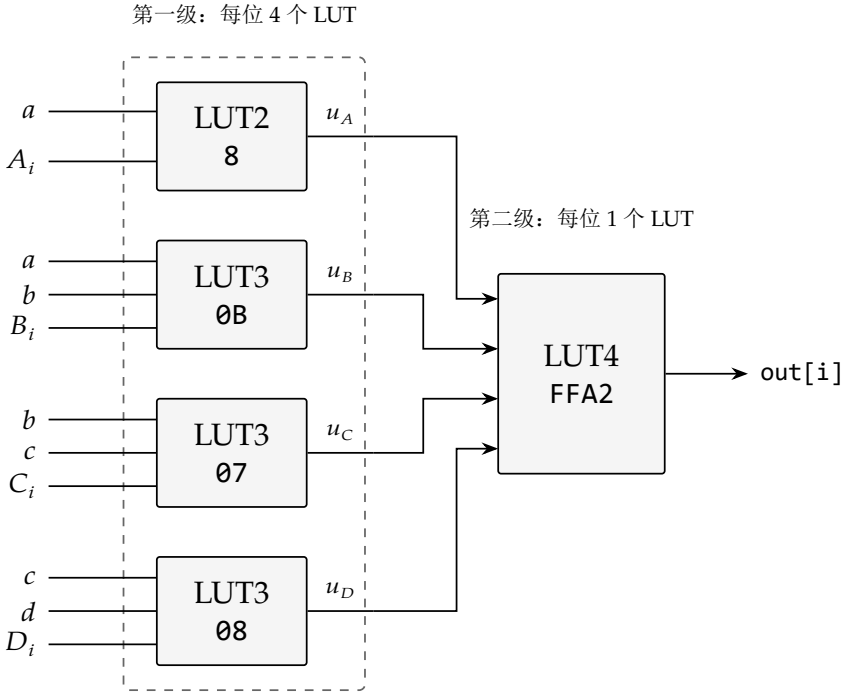


图 2.4　test2 原版的四级优先 MUX 链

未经 P2 优化时，test2 使用原版 ABC9 命令流映射得到 25 个 LUT，LUT 深度为 2。对于其 5 位输出，平均每位需要 5 个 LUT。每位输出对应的 5 个 LUT 有效输入数分别为 2、3、3、3、4，表明当前映射仍存在重新组织逻辑、减少 LUT 数量的空间。



$i = 0, \ldots, 4$，五个位切片相同：A_i = `dataa[i]`，其余类推
总计 25 个物理四输入 LUT，LUT 深度为 2

图 2.5 test2 原版 ABC9 映射后的 LUT 网络

2.3.1 P2 的优化方法

P2 沿 MUX 的 A 输入端反向追踪，识别位宽一致、选择信号均为单比特的四级优先链，然后重组其中优先级最高的两级。用 A, B, C, D 表示四路数据、E 表示默认输入，一般形式及其变换为：

$$\underbrace{Y = a?A : (b?B : (c?C : (d?D : E)))}_{\text{原优先选择链}} \quad \Longrightarrow \quad \begin{cases} H = a?A : B, & L = c?C : (d?D : E), \\ Y' = (a \vee b)?H : L. \end{cases}$$

其中 E 可以是常量、独立信号或其他组合逻辑的输出；特别地，test2 中为常量 $E = 0$。

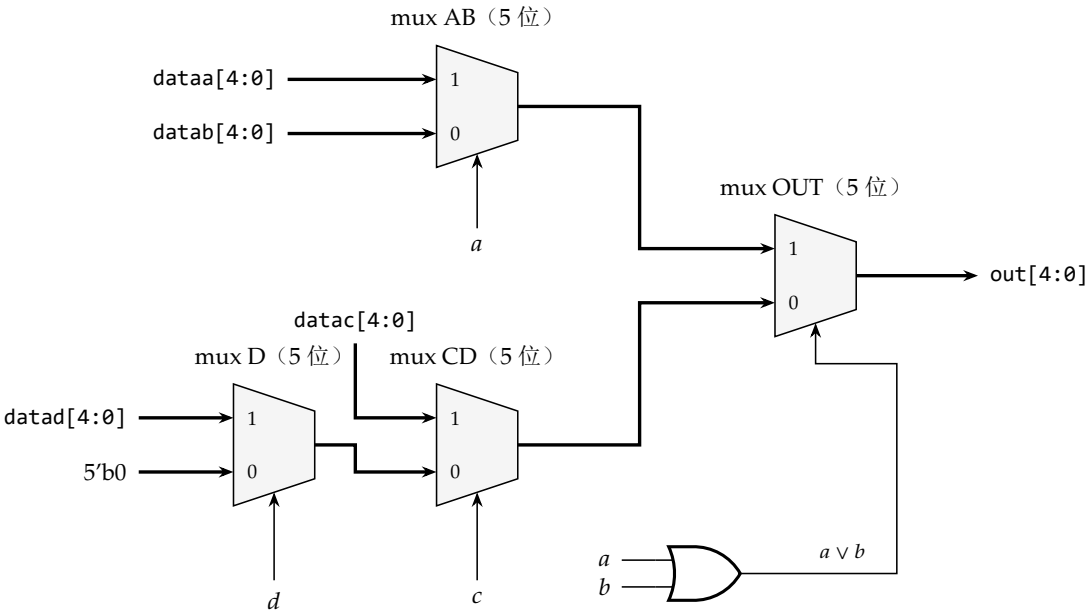
重组后，高优先级数据与低优先级数据可以并行计算，再由 $a \vee b$ 控制最终选择。

对于 test2 的默认零结构，P2 重组后，ABC 可以进一步将逻辑组织为两个并行的数据函数及一个汇总函数，每个函数均可由一个四输入 LUT 实现。

实测 test2 映射后的 LUT 数由 25 个减少至 15 个，降低 40%；平均每位输出需要 3 个 LUT，LUT 深度仍为 2，每位输出对应的 3 个 LUT 有效输入数均为 4。在保持现有 LUT 深度的同时，减少了所需的逻辑资源。

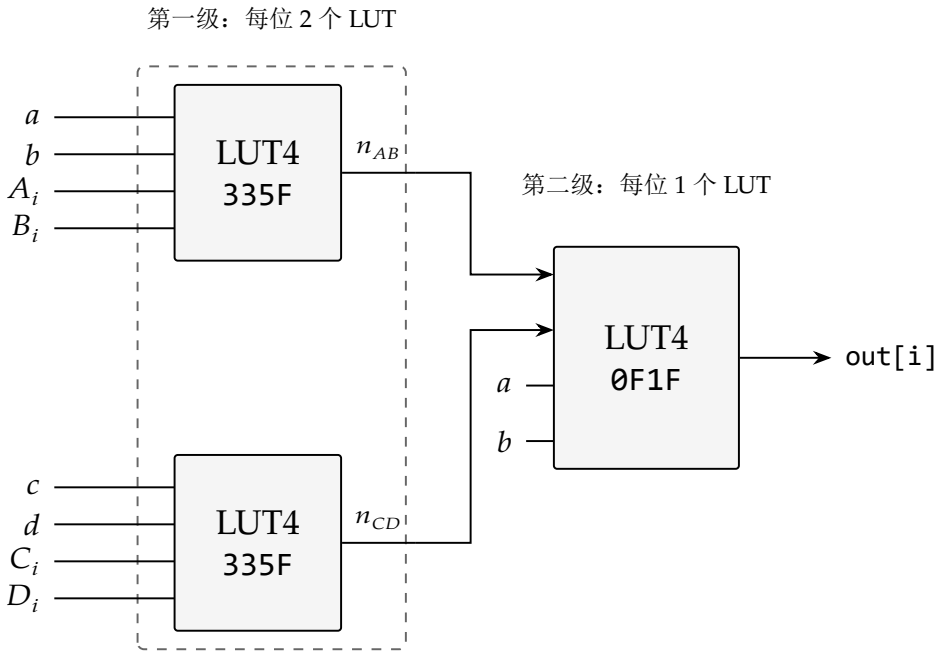
2.3.2 结构识别、网表改写与正确性

实现入口为 `run_priority_balance_rule()`。规则先建立 MUX 输出到驱动单元的索引，再从被选中的顶层 MUX 沿 A 端查找连续四级结构；每一级均检查数据位宽及



4 个 5 位二选一 mux + 1 个共享二输入 OR 门
优先级 $a > b > c > d$

图 2.6 test2 经 P2 重组后的 MUX 结构



$i = 0, \ldots, 4$，五个位切片相同；A_i = `dataa[i]`，其余类推
总计 15 个物理四输入 LUT，LUT 深度为 2
中间反相与 $a \vee b$ 均由 LUT 真值表实现，无独立反相器或 OR 门

图 2.7 test2 优化后 ABC9 映射的实际 LUT 结构

单比特选择端，且要求驱动输出与所追踪的整条信号一致，避免把部分位连接误判为完整链。

改写时，新建高优先级数据 MUX、共享控制信号 $a \vee b$ 及最终 MUX，并将最终结果接到原顶层输出。低优先级部分直接复用原第三级 MUX 的输出；中间旧节点不在本规则中强制删除，因此其余使用者仍可取得原值。被删除的顶层节点会登记到已改写集合，后续索引查询跳过这些节点，避免访问失效对象；后续清理再移除无用逻辑。

等价性可按控制条件分为两种情况。当 $a \vee b = 1$ 时，若 $a = 1$，两种结构均输出 A；否则必有 $a = 0, b = 1$，两种结构均输出 B。当 $a \vee b = 0$ 时，$a = b = 0$，两种结构均输出 L。因此变换保留原优先级，对任意数据及默认输入均成立。

补充案例覆盖默认输入变化、中间结果复用及更长选择链；其中复用中间结果可能改变面积与深度的取舍。测试数据及分析见第 3.3.2 节。

2.4 P3 比较逻辑共享与使能条件利用

P3 面向常量比较、寄存器使能和写入数据选择共同构成的控制逻辑。设计思路是先提取使能条件提供的取值信息，用于简化数据端和复用控制信号，再对剩余常量比较统一规划共享。比较对象和约束均从网表连接中识别。

处理顺序与实现一致：首先由 `EnableDcRewriter::run()` 分析带使能寄存器，将写入条件下等价的数据选择替换为已有信号位；随后由 `reuse_enables()` 复用已有使能，减少重复判断；最后由 `ComparePlanner::run()` 共享剩余比较中的公共位条件。前两步依赖原始使能中的常量比较关系，因此先利用这些信息，再重组比较网络。

下面先给出 test3 的输入结构，再按上述顺序说明各项通用处理及其在案例中的应用。

2.4.1 优化对象与案例结构

test3 包含 8 位移位寄存器 `sr`、8 位输出寄存器 `gpioout` 和 1 位寄存器 `sr_out`。当 `cs=0` 时执行移位，`gpioout` 保持；当 `cs=1` 且 `sr[7]=1` 时，按命令写入相应输出位。记三个比较结果为

$$A = [sr = \texttt{8'hAA}], \quad B = [sr = \texttt{8'hFF}], \quad C = [sr = \texttt{8'hFE}].$$

下文电路图仍使用 AA、FF、FE 标注这三个信号。过程转换和通用优化将输出分成三段，各段的写入条件为：

表 2.2 test3 三段输出的使能与写入关系

寄存器段	使能条件	写入来源
`gpioout[7:4]`	$E_h = cs \wedge sr[7] \wedge (A \vee C)$	AA、FE 两个分支的数据
`gpioout[3:1]`	$E_l = cs \wedge sr[7] \wedge (A \vee B)$	AA、FF 两个分支的数据
`gpioout[0]`	$E_0 = cs \wedge sr[7] \wedge \neg C$	AA、FF 和默认分支的数据

其中默认分支只写 `gpioout[0]`，其余位保持。`opt_dff` 将保持关系提取为使能，得到三个 `$dffe`；使能无效时，对应 D 端可以是未定义值，因为此时不发生写入。图 2.8 展示 P3 入口处的组合逻辑及使能连接，保留了通用优化已经完成的常量简化。

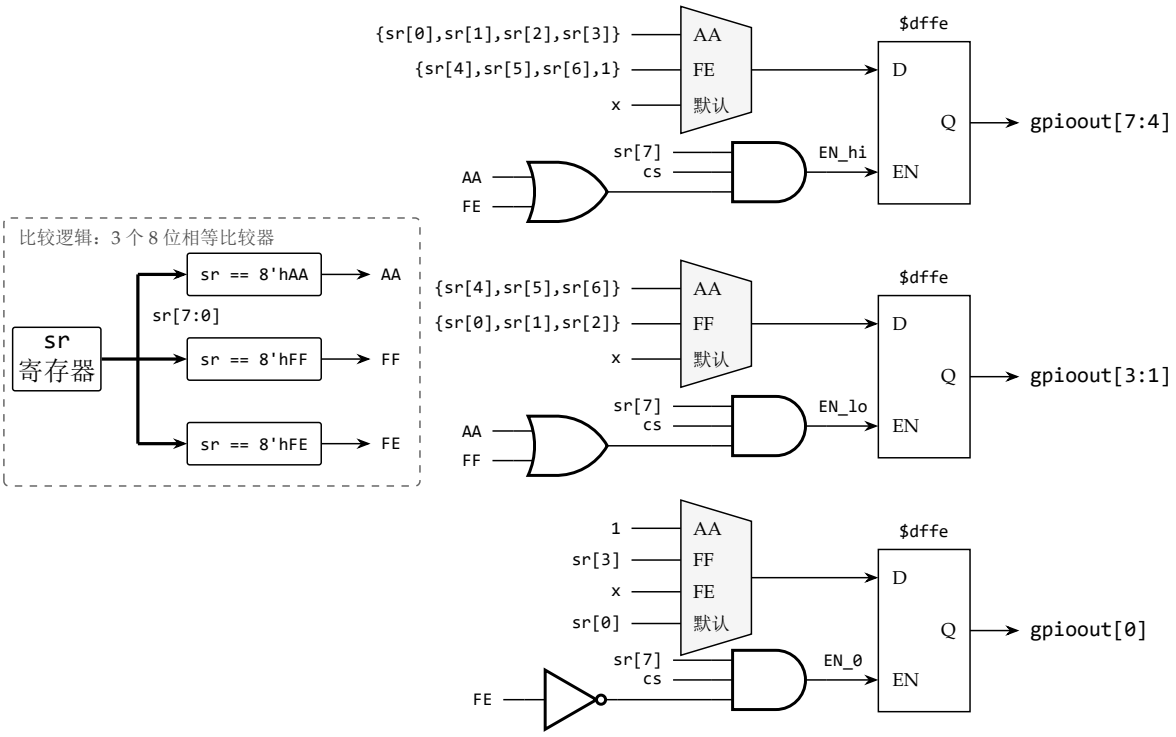


图 2.8 test3 在 P3 改写前的比较、使能与数据选择结构

2.4.2 使能约束下的数据端化简

该步骤沿寄存器使能的组合逻辑识别与常量比较的信号 s，利用其可能写入的取值范围化简 D 端。

带使能寄存器的行为是：

$$Q^+ = \begin{cases} D, & E = 1, \\ Q, & E = 0. \end{cases}$$

若只修改 D 端而保持使能，则只需满足 $E = 1 \Rightarrow D' = D$，不必要求使能无效时两个组合数据函数也相同。这一约束直接来自寄存器的写入语义。

`EnableDcRewriter::run()` 从使能逻辑中识别比较信号，并用三值求值求出使能

可能有效时的取值集合 V。比较信号以外的输入视为未知，只排除使能确定为 0 的取值，因此 V 是保守范围。只有原 D 端在 V 内的值全部确定，且逐项等于某个比较信号位时，才接到该信号位：

$$\forall v \in V: \quad D(v) = s_j(v) \quad \Longrightarrow \quad D' = s_j.$$

替换候选限定为已有信号位，不直接接常量，以避免后续常量寄存器删除改变映射初态模型。初态与复位行为纳入整设计等价检查。

以 `gpioout[7]` 为例，其使能为 E_h；使能有效时，`sr` 只能为 AA 或 FE。原写入值在 AA 时为 `sr[0]`，即 0；在 FE 时为 `sr[4]`，即 1。`sr[2]` 在两种命令下也分别为 0 和 1，因此可直接将这一 D 端接到 `sr[2]`。

2.4.3 通过区分位复用已有使能

数据端分析同时记录了各使能对应的被比较信号和可能取值集合。此步骤在目标使能的逻辑锥中寻找单独使用的常量比较 $[s = k]$，再查找同样依赖 s 的另一使能 E_s。若 E_s 有效时的候选取值中，某一位能够将 k 与其他取值区分，就尝试用 E_s 与该位的匹配条件构造替代信号，并接回目标使能的原比较位置。

候选筛选要求被替换位置满足单使用者条件，且被复用使能的候选取值不超过四个。随后排除组合环，并对目标使能与被复用使能的全部输入进行真值核对；合并支撑集最多为 12 位。只有目标使能在全部输入组合下均确定且保持相同，才采用替换。区分位用于生成候选，完整函数核对决定是否改写。

在 test3 中，E_h 只在 AA 或 FE 命令下有效，而 `sr[2]` 能区分二者：AA 时为 0，FE 时为 1。因此：

$$E_h \wedge sr[2] = cs \wedge C, \qquad E_0 = cs \wedge sr[7] \wedge \neg(E_h \wedge sr[2]).$$

第二个等式利用了 FE 必有 $sr[7] = 1$。改写后的 E_0 只需输入 $cs, sr[7], E_h, sr[2]$，可以由一个四输入 LUT 实现；原来只为这一使能保留的 FE 部分比较有机会被删除。

2.4.4 常量比较的共享规划

前两步完成后，将剩余的常量比较按被比较信号归组，对每组建立公共部分比较和各自剩余条件的共享网络。

这一过程由 `ComparePlanner` 统一规划。它将同一输入信号上的常量比较归为一组，以“位置掩码、该位置上的目标值”表示部分比较。候选来自整组公共位、常量取值列具有相同模式的位置，以及两常量之间的公共位；较大的位置集合再产生适合四输入 LUT 的候选。规划器将候选和各比较剩余位构造成同一共享图，统一评估节点复用与深度。

候选方案依次比较节点总数、最大估计深度及各根深度之和，仅保留成本优于当前方案的选择；重复节点通过缓存复用。每个比较根在生成前均检查其位置覆盖完整、目

标值与原常量一致。候选掩码数、搜索轮数及工作量均设有上限，以限制大比较组的分析成本。

对 n 位信号 s 与常量 k 的相等比较，可以写成各位匹配条件的合取：

$$[s = k] = \bigwedge_{j=0}^{n-1} \ell_j, \qquad \ell_j = \begin{cases} s_j, & k_j = 1, \\ \neg s_j, & k_j = 0. \end{cases}$$

不同常量若在某些位置取值相同，就可以复用这些位置上的部分比较；连续或离散的位置均可共享。test3 的三个常量在奇数编号位上均为 1：

表 2.3 test3 比较常量的公共位条件

常量	第 7、5、3、1 位	第 6、4、2、0 位
AA	1111	0000
FF	1111	1111
FE	1111	1110

因此，令 $N = sr[7] \wedge sr[5] \wedge sr[3] \wedge sr[1]$，三个完整比较可写为：

$$A = N \wedge \neg sr[6] \wedge \neg sr[4] \wedge \neg sr[2] \wedge \neg sr[0],$$

$$B = N \wedge sr[6] \wedge sr[4] \wedge sr[2] \wedge sr[0],$$

$$C = N \wedge sr[6] \wedge sr[4] \wedge sr[2] \wedge \neg sr[0].$$

四个奇数位的检查由公共节点 N 实现一次，再与各自的偶数位条件组合。后续 ABC9 还可将偶数位条件与下游使能合并，直接复用部分比较的结果。

2.4.5 改写结果与正确性

图 2.9 汇总三项处理后的连接：写入数据直接复用信号位，使能使用已有控制结果，剩余比较共用公共位条件。

三项改写分别具有对应的正确性条件：比较共享保持完整的组合比较函数；数据端替换保持所有可能写入条件下的 D 值；使能复用保持整个使能函数。数据端接线不直接修改寄存器参数，后续只有 D 端、时钟、使能、初态及其他控制条件兼容的寄存器才可合并。最终使用三组整设计等价检查验证包含初态与复位的完整行为。

规则根据网表中的比较关系和写入条件识别候选；遇到不支持的单元、未知求值结果或超出分析预算的情况时跳过改写。各处理步骤的消融结果见第 3.3.3 节。

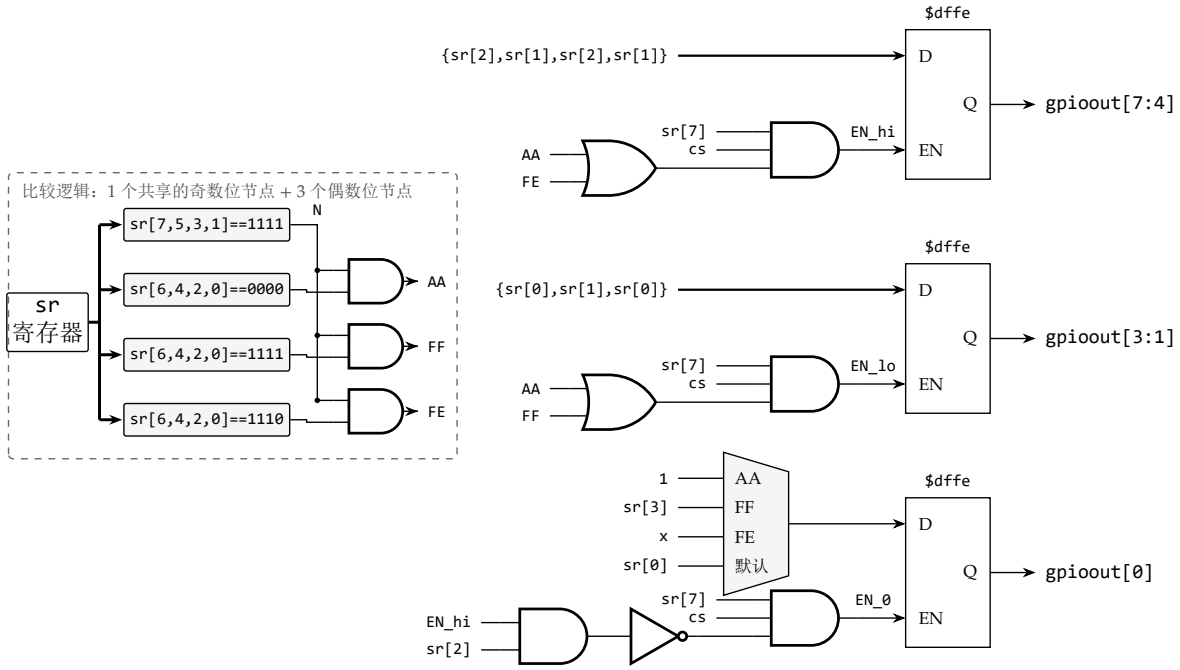


图 2.9　test3 经 P3 改写后的结构：共享部分比较、数据位直连及使能复用

2.5　P4　小规模查表逻辑分解与共享

P4 面向块 RAM 映射后保留的小规模异步 ROM，将逐位查表函数分解为四输入 LUT 网络，并在不同输出之间共享相同或互补的中间函数。[3]

以 test4 为例，其常量表为 32 × 10 bit。虽然设计包含输出寄存器，但读方式、使能及复位连接未满足当前 M9K 映射条件，原流程最终通过 LUT 实现查表部分。P4 保留原有时序控制，仅重新组织这一组合查表网络。

2.5.1　ROM 检查与输出函数提取

`run_rom_decompose_rule()` 遍历网表中的 `$mem_v2` 单元，筛选出符合条件的 ROM 结构：地址位宽为 1～5 位、输出位宽为 1～64 位，具有完整且确定的初始化数据，无写端口，仅有一个异步读端口，并满足读使能恒有效、无读端口复位等条件。

对于符合条件的 ROM，代码按输出位读取初始化数据，将每个输出分别提取为一个布尔函数。以 test4 为例，32×10 bit 的查表结构被表示为 10 个五输入函数，每个函数描述一个输出位在全部 32 种地址下的取值。

随后调用 `normalize_rom_function()` 进行化简。如果某个地址位不影响输出，就将其从函数输入中删除；对于相同或互补的函数，采用统一的表示方式，以便后续共享。

2.5.2　五输入函数的两类分解

化简后，常量可以直接连接，单输入函数可以由地址信号或其反相实现，2～4 输入函数则最多需要一个四输入 LUT。因此，依赖 5 个输入的函数需要进一步分解。

P4 考虑以下两类实现方式。

按地址位展开。选取一个地址位作为选择信号，分别固定其为 0 和 1，得到两个四输入子函数。例如：

$$F(x_0,x_1,x_2,x_3,x_4) = x_4\,?\,F_1(x_0,x_1,x_2,x_3) : F_0(x_0,x_1,x_2,x_3).$$

其中，F_0 和 F_1 由 `rom_cofactor()` 得到，各自最多用一个 LUT 实现，再用一个三输入选择函数恢复原函数。因此，一般需要两个叶节点 LUT 和一个汇总 LUT；若子函数为常量、只依赖单个输入，或与其他输出的子函数相同，实际数量还可以减少。P4 尝试不同的分解地址位，并识别不同输出之间相同或互补的子函数，避免重复实现。这一方式作为基础方案，也为后续搜索提供初始候选。

通过中间信号进行编码分解。这一方案将部分地址信息编码为一个中间信号 g，输出函数再结合 g 与其余地址位计算结果。两级函数分别限制在四个输入以内，从而用两个 LUT 实现原五输入函数。

将五个地址位划分为两个待编码变量 $B = (b_0, b_1)$ 和三个直接变量 $D = (d_0, d_1, d_2)$，再从 D 中选取至多两个变量组成共享集合 S，其余变量记为 $U = D \setminus S$。候选结构为：

$$g = G(B,S), \qquad F = H(g,D).$$

G 的输入数为 $2 + |S| \le 4$，H 的输入数为 $1 + 3 = 4$，因而每个函数最多需要一个四输入 LUT。

判断编码是否可行时，先固定共享变量 S 的一个取值。依次令 B 取 00、01、10、11，得到四个仅关于 U 的子函数；每个子函数的真值表构成一列。若这四列最多只有两种不同模式，就用 $g = 0$ 和 $g = 1$ 区分这两类，第二级 H 根据 g 和 D 恢复输出。对 S 的每个取值都检查这一条件，随后枚举编码极性，并对全部 32 种地址核对生成结果。

test4 的具体例子。令 x_i 表示地址输入 `datain[i]`。test4 的十位查表输出由六位 `dataout` 和 `only_5b`、`datain_5bis7`、`data_dp`、`data_dn` 四个标志组成。每一位分别建立候选，随后统一选择；下面用其中的 `data_dp` 说明分解。

`data_dp` 仅在地址 17、18、20 时为 1。这三个地址的高两位均为 $x_4x_3 = 10$，低三位分别为 001、010、100，因此可写成：

$$g_1 = \bar{x}_2\bar{x}_1x_0 \vee \bar{x}_2x_1\bar{x}_0 \vee x_2\bar{x}_1\bar{x}_0,$$
$$F_{\mathrm{dp}} = x_4\bar{x}_3g_1.$$

这里 g_1 是一个三输入函数，第二级也只有三个输入，共两个 LUT。这一例子说明五输入查表如何转成两级 LUT；其余九位同样建立候选并参与联合选择。上述两类候选均

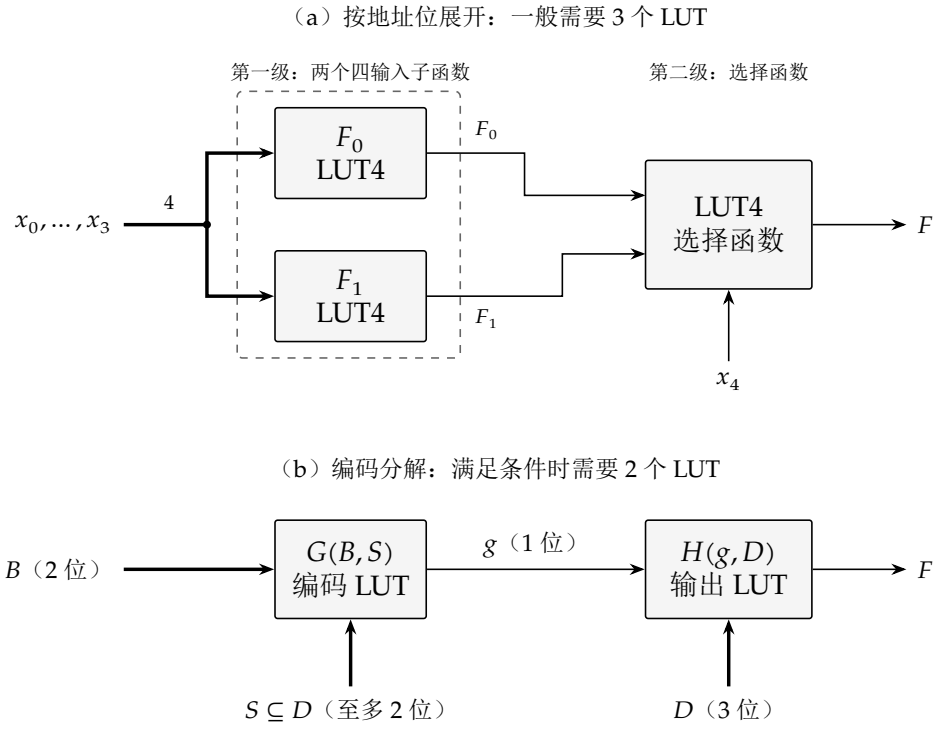


图 2.10　五输入函数的两类 LUT 分解结构

由 `RomLutPlanner::options()` 构造。候选生成过程中会消除无关输入、合并相同函数，并逐项检查候选真值表与原函数是否一致。对于五位地址 ROM，化简后不足五输入的输出也纳入同一候选体系，以便与其他输出共同评估共享效果。

2.5.3　跨输出成本评估与方案选择

P4 对整个 ROM 联合评估，将各输出的分解方案与中间函数共享一并计入成本。

因此，`RomLutPlanner::plan()` 将各输出候选组织为共享函数网络，按照所有输出实际使用的不同节点统计成本。相同节点被多个输出引用时，只计入一次。方案优先比较 LUT 节点总数，数量相同时再比较输入引脚总数。

例如，若输出 $F_i = H_i(g, A)$ 与 $F_j = H_j(g, B)$ 使用同一个中间函数 g，分别实现需要两组两级网络；共享后只生成一个 g 节点和两个输出节点。共享比较同时检查实际输入连接与真值表。互补函数采用统一表示，并在引用处记录极性。

设 ROM 有 m 个输出，选择方案为 $P = (p_0, \ldots, p_{m-1})$；记第 i 个输出所用的 LUT 节点集合为 V_{i,p_i}，联合面积估计为：

$$C_{\mathrm{LUT}}(P) = \left| \bigcup_{i=0}^{m-1} V_{i,p_i} \right|.$$

test4 中 $m = 10$，上述并集使跨输出复用直接计入整体成本。

搜索先比较五个地址位展开的初始方案，再逐输出替换候选，每次只保留全局成本

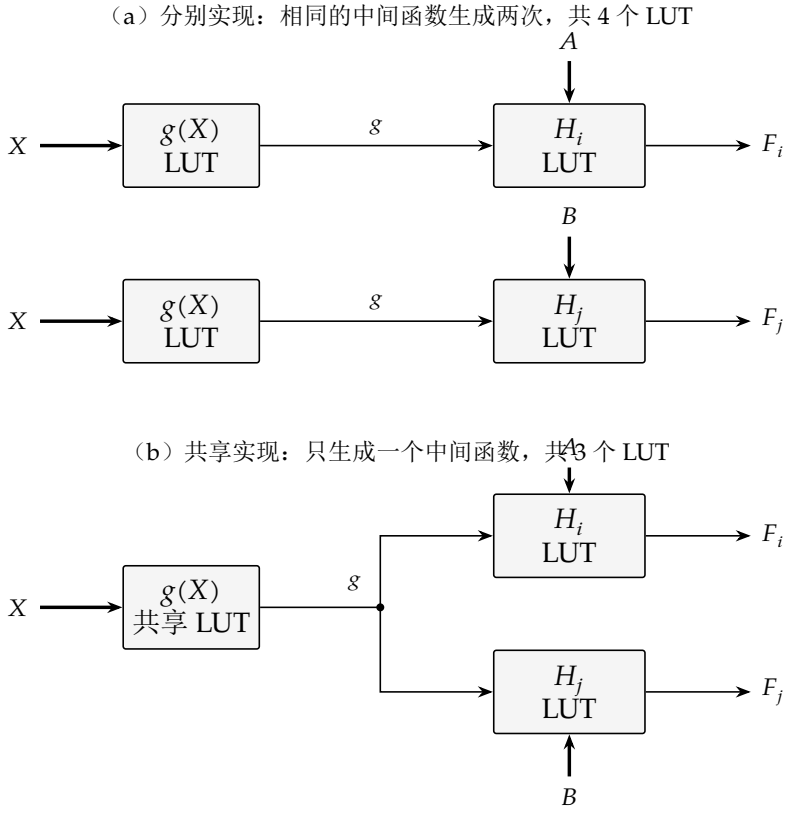


图 2.11　跨输出复用中间 LUT 的结构示意

下降的选择；最多进行八轮扫描，若一轮没有变化则提前停止。

获得联合方案后，主函数再调用 `rom_plan_cost()`，计算基础展开方案的最低估计成本，并与联合方案比较。只有联合方案估计至少节省 2 个 LUT，且节省量不低于基础成本的约 1/8，才采用该方案；否则使用基础方案。

对于地址位宽为 1～4 位的 ROM，不启动上述五输入候选搜索，直接对化简后的函数进行共享成本统计和电路生成。

2.5.4　网表改写与正确性

方案确定后，P4 根据选中的函数网络生成电路。常量及地址信号直接连接，其他函数生成相应的逻辑 LUT；已经生成的共享节点直接复用，需要的反相信号也通过缓存避免重复生成。

最后，将新网络连接到原 ROM 的输出端，并删除原 `$mem_v2` 单元。改写范围仅限组合查表逻辑，后续寄存器、使能、复位及其他使用者保持原有连接。

对确定的初始化数据，原 ROM 的每个输出就是地址的一个确定布尔函数。无关输入消除不改变真值表；地址位展开由余因子恒等式保证正确；编码候选在全部地址上逐项核对。节点共享仅复用同一实际输入上的同一函数，因而保持每个输出值。未满足读端口、初始化及规模限制的 ROM 不进入本规则；原有寄存器、使能、初态和复位语义

均不改写。

电路生成由 `RomEmitter` 等生成器完成，共享节点及反相信号均通过缓存复用。

保持原版 ABC9 命令流、单独启用 P4 的映射对照见第 3.3.3 节。

2.6 P5 完整译码选择网络区域重组

P5 面向两类选择网络：一类是完整地址译码与并行多路选择器构成的局部结构；另一类是多个 PMUX 层叠、且内部子节点被不同父节点共用的选择区域。前者将地址先展开为独热条件，再完成数据选择；后者还需要沿多层网络传递宽数据。P5 分别恢复按地址位控制的二进制 MUX 树，以及按最终数据组织的单层 PMUX，使后续 ABC9 能够在重新组织的网表上映射。

2.6.1 两类选择结构及其映射冗余

完整译码与并行选择： 设三位地址 $a = (a_2, a_1, a_0)$ 控制一条 32 位数据通路，八个码值对应的数据依次为

$$(D_0, D_1, D_2, D_3, D_4, D_5, D_6, D_7) = (A, B, C, D, A, B, C, D).$$

其中 A, B, C, D 为四条不同的 32 位数据总线。该结构用七个相等比较生成 PMUX 的选择条件，地址 7 对应默认输入，如图 2.12。这个结构映射出的网表会明确表达八个选择分支，却不会体现地址位的层次选择关系。若译码与选择被分开处理，数据通路仍可能携带重复分支及不必要的选择层级。在此例中，输出实际上与 a_2 无关；而我们的改写正是从完整数据表中恢复这种关系。

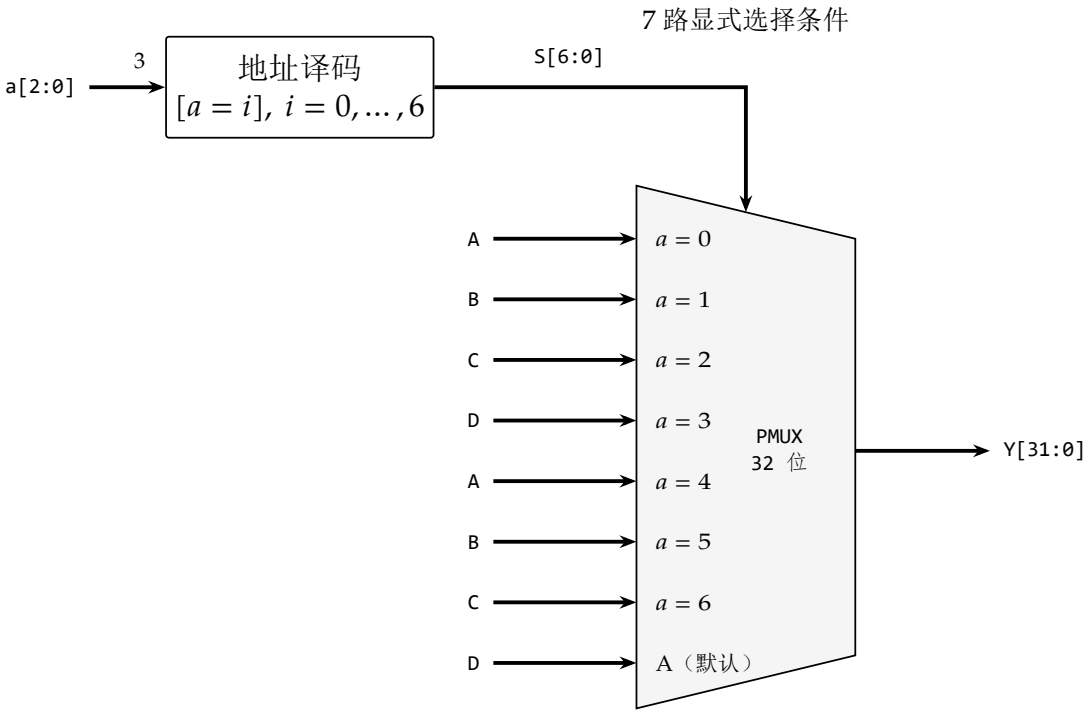
具有共享子节点的多层 PMUX： 另一例由四个四路选择节点 R, A, B, C 构成，四个地址 r, a, b, c 均为两位，数据宽度均为 32 位。本例的 A, B, C 表示选择节点，而 $D_0, D_1, D_2, P, Q, U, V$ 表示最终数据。各节点按地址 0 至 3 的输出表为

$$R(r) = (A, B, U, V), \quad A(a) = (C, P, Q, P),$$
$$B(b) = (Q, C, P, Q), \quad C(c) = (D_0, D_1, D_0, D_2).$$

其中，节点 C 同时被 A 和 B 使用，形成图 2.13 的菱形共享关系；P, Q, D_0 还分别从多条路径出现。选择 D_i 时数据需经过 C、A 或 B、R 三层选择。逐层保留会限制跨节点汇总路径与相同数据的机会；若按树展开又可能复制共享子结构。P5 以整个选择区域为处理对象，在保留共享关系的同时汇总路径条件，并按最终数据重组选择网络。

2.6.2 完整译码结构的二进制 MUX 树重建

完整译码使每个地址值都对应一份确定数据。P5 先将显式分支与默认分支整理为完整表 D_v，再按地址位递归构造二选一 MUX。若两支连接相同，直接复用该信号，省



数据总线均为 32 位；地址 7 对应默认输入 D。
地址 0/4、1/5、2/6、3/7 分别选择同一份数据。

图 2.12 完整译码控制的八路选择实例：不同地址重复选择同一数据总线

去当前 MUX。

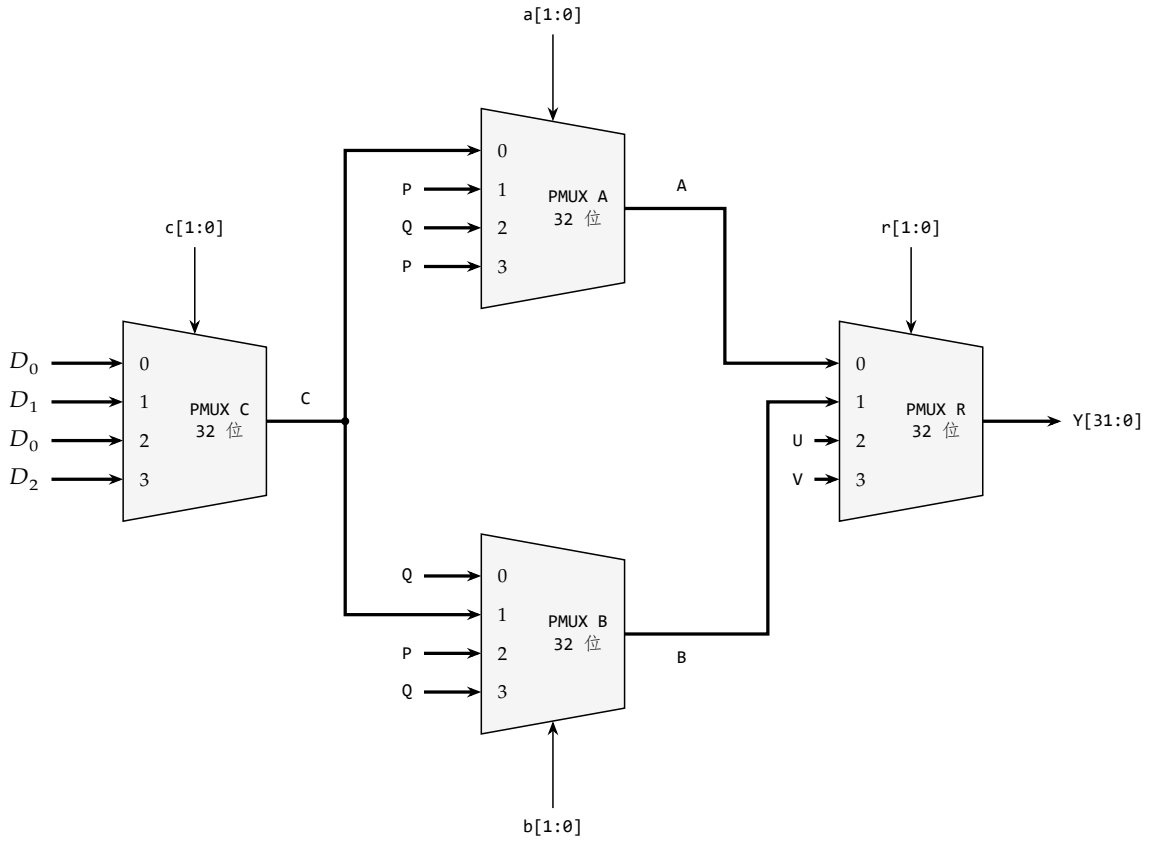
图 2.12 中，地址 0 与 4 均取 A，1 与 5 均取 B，2 与 6 均取 C，3 与 7 均取 D，因此 a_2 不影响输出。按 a_0、a_1 的层次整理后，仅需三个 32 位 MUX：

$$L = \mathrm{mux}(a_1, A, C), \quad H = \mathrm{mux}(a_1, B, D),$$
$$Y = \mathrm{mux}(a_0, L, H).$$

`PmuxCompleteDecoderRegion::run()` 用 `SigMap` 归一化连接，识别同一地址上的相等比较与零值检测。候选限于 2 或 3 位地址，各显式条件须对应不同码值，默认输入补齐唯一遗漏值。`Choice` 按码值整理数据；`emit(level,code)` 按固定地址位顺序递归生成 MUX，并在两支相同时复用已有信号。

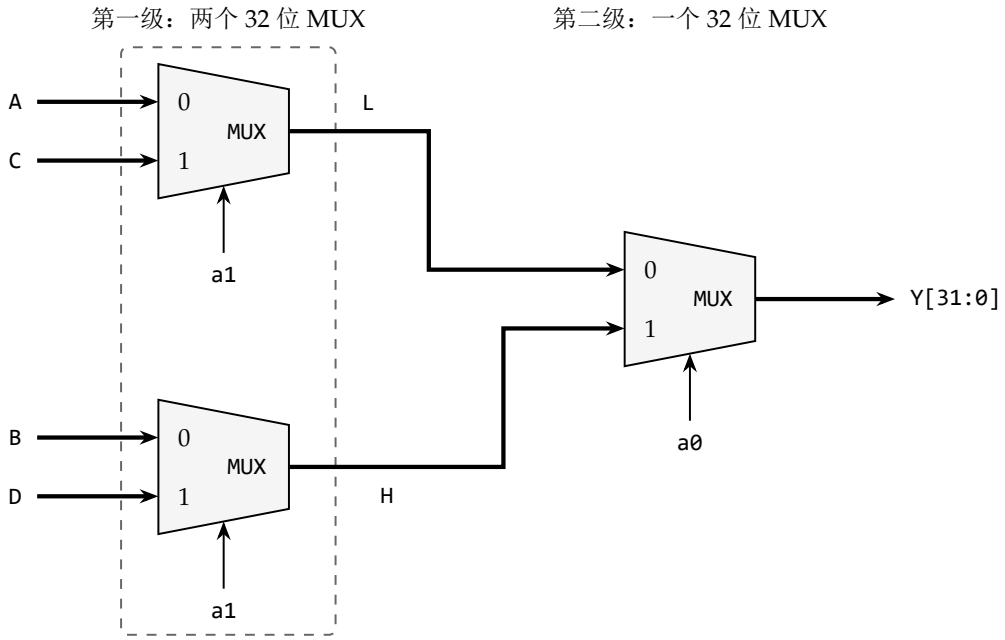
2.6.3 共享区域的路径条件合并与单层选择重建

改写原理： P5 从区域根输出出发计算各份最终数据的选通条件：同一路径上的分支条件取与，到达同一节点或同一数据的多条路径取或。共享子节点先汇总全部进入条件，再向其输入传播。最终，将每份不同数据及其选通条件接入一个单层 PMUX。区域内各层均为完整译码选择，因此确定地址下仍恰好选中一份数据。



数据总线均为 32 位；节点按完整地址选择表表示，省略各节点的地址译码。
端口编号为地址值，3 为默认分支；实心分叉点表示 C 同时供 A、B 使用。

图 2.13　多层 PMUX 的共享实例：两个父节点共用子节点，且最终数据重复出现



数据总线均为 32 位；a0、a1 为单比特选择信号。
地址位 a2 不影响输出，已在重建时消去。

图 2.14　八路实例重建后的选择树：地址位 a_2 被消去

实例中的条件合并： 图 2.13 中，根节点 R 在 $r=0,1$ 时分别选择 A,B，在 $r=2,3$ 时直接选择 U,V。共享节点 C 可沿 $R\to A\to C$ 或 $R\to B\to C$ 被选中，因此其总选通条件为

$$H=((r=0)\wedge(a=0))\vee((r=1)\wedge(b=1)).$$

接着分析 C 内部的数据选择。节点 C 在 $c=0$ 或 $c=2$ 时均输出 D_0，因此最终选择 D_0 的条件是 $S_{D_0}=H\wedge((c=0)\vee(c=2))$。类似地，$C$ 在 $c=1$ 时输出 D_1、在 $c=3$ 时输出 D_2，对应的最终选择条件分别为 $H\wedge(c=1)$ 和 $H\wedge(c=3)$。这样，原来需要先经过 R、再经过 A 或 B、最后经过 C 才能选中的数据，就可以直接由合并后的条件控制。

对重复出现的数据 P,Q 同样合并路径条件：

$$S_P=((r=0)\wedge((a=1)\vee(a=3)))\vee((r=1)\wedge(b=2)),$$
$$S_Q=((r=0)\wedge(a=2))\vee((r=1)\wedge((b=0)\vee(b=3))).$$

根节点直接连接的 U,V 分别由 $r=2,3$ 选通。

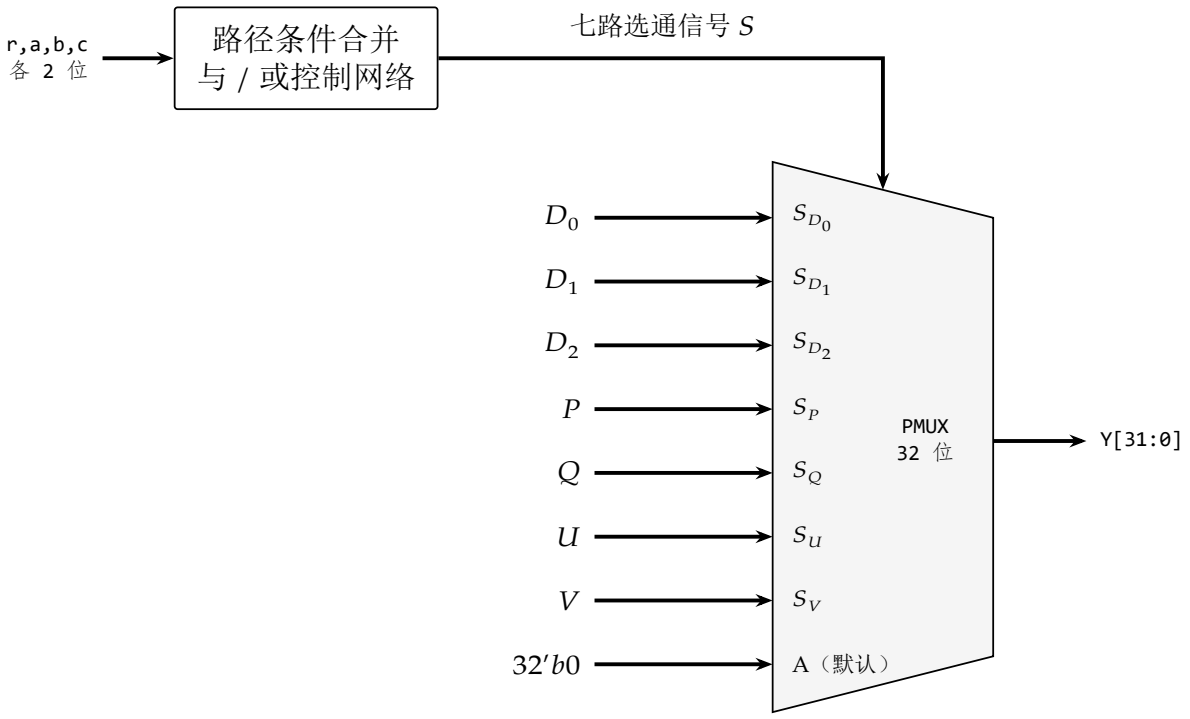
最终，P5 将 D_0,D_1,D_2,P,Q,U,V 七份不同数据各接入新 PMUX 的一路，并使用上述合并条件作为对应的选择信号，如图 2.15。例如，$r=0,a=0,c=2$ 与 $r=1,b=1,c=0$ 原本沿不同路径选中 D_0，现在均由同一个选择信号 S_{D_0} 覆盖。改写后，数据直接通过新的 PMUX 到达输出，原来多层选择的作用则由控制网络中的与、或条件表达。

代码实现： 实现首先沿候选 PMUX 的数据连接收集共享区域，并检查内部节点是否仍有区域外使用者。通过检查后，代码从根节点向下计算选择条件：用 `en` 记录每个子节点何时被选中，用 `terms` 记录每份最终数据何时成为输出。同一子节点或同一数据存在多条到达路径时，将这些条件取或。处理顺序保证父节点先于子节点，使共享子节点能够先收齐所有进入条件。

这些条件先由 `BoolPlan` 记录为临时逻辑图，不立即修改网表。已有译码信号直接复用，默认分支则补建对应遗漏地址值的比较；相同的比较和与、或运算通过缓存共享。随后，代码分别估计原区域和新结构的数据选择开销，并将新增控制逻辑计入新结构成本。只有新估算严格小于原估算时，才调用 `BoolPlan::emit()` 生成控制单元，再创建一个以最终数据为输入、以合并条件为选择信号的新 PMUX，并接回原根输出。区域还受到无环、外部使用、搜索规模和最终数据数量等限制。

2.6.4 网表结构与安全性

P5 保留原输出连接和数据位序，并将原默认输入作为遗漏地址值对应的数据参与改写。共享区域中新 PMUX 的默认零仅作占位：对于确定的二进制地址，合并后的选择条件始终覆盖原有分支，因此不会用零替代原默认数据。代码跳过译码关系不明确的



控制网络生成的条件（与数据端口同名对应）

$H = R_0A_0 \vee R_1B_1$

$S_{D_0} = H(C_0 \vee C_2)$　　$S_P = R_0(A_1 \vee A_3) \vee R_1B_2$

$S_{D_1} = HC_1$　　$S_Q = R_0A_2 \vee R_1(B_0 \vee B_3)$

$S_{D_2} = HC_3$　　$S_U = R_2$，　$S_V = R_3$

$R_i = [r = i]$，$A_i = [a = i]$，$B_i = [b = i]$，$C_i = [c = i]$；并列表示逻辑与。
七份数据均为 32 位；确定地址下恰好一路有效，默认零仅作占位。

图 2.15　共享区域重组结果：合并到达条件，以七份不同数据构成单层 PMUX

候选，并检查共享区域是否存在组合环、内部结果的区域外使用及规模超限等情况。改写仅调整组合选择网络，不修改寄存器及其初态、时钟和复位参数；整设计正确性由第 3.2 节的等价性验证核对。

实现先完成区域分析和新网表生成，再删除待替换的旧单元，避免分析过程中访问失效对象，并调用 `opt_clean` 清理无用逻辑。

2.7　ABC9 候选映射与面积优先选择

ABC9 阶段以经过 Pass 优化的同一份电路为起点，分别执行“重写后映射”和“入口逻辑直接映射”，生成两路候选并按面积选取结果 [4-5]。

2.7.1 默认命令流与调整位置

`synth_intel` 的 LUT 映射阶段执行 `abc9 -lut 4 -W 300`。默认命令流定义在 Yosys 的 `abc9.cc` 中，由 `abc9_exe.cc` 生成供外部 ABC 执行的脚本。本工程通过调整该命令流组织两路映射候选。原命令为：

```
&scorr; &sweep; &dc2; &dch -f -r;
&ps; &if {W} {D} {R} -v; &mfs
```

前四项完成等价归并、清理及逻辑重写，`&if` 根据 LUT 和延迟模型完成映射，`&mfs` 原本用于映射后的进一步逻辑优化。`{W}` 等是 Yosys 替换的参数占位，其中本流程的 W 对应 `-W 300`；未指定的 D、R 参数为空。`&ps` 打印统计。

优化版采用下述两路候选命令流。

2.7.2 两个映射候选的生成

优化后的默认命令流为：

```
&saveaig;
&scorr; &sweep; &dc2; &dch -f -r;
&if {W} {D} {R} -C 16 -v; &save -a;
&loadaig; &if {W} {D} {R} -v; &save -a;
&load; &ps -l
```

候选一：重写后映射。先用 `&saveaig` 保存 ABC 入口 AIG，再执行与默认流程相同的主要逻辑重写，随后通过 `&if -C 16` 映射。C 指定每个节点保留的候选 cut 数量上限，当前配套版本默认值为 8；提高到 16 扩大覆盖选择空间。

候选二：直接映射。`&loadaig` 恢复先前保存的入口 AIG，跳过前述重写，使用默认 cut 上限完成映射。这一路保留了 Pass 输出的结构关系，可以与重写后产生的不同覆盖进行比较。两路均使用相同的四输入 LUT 约束及延迟相关参数。

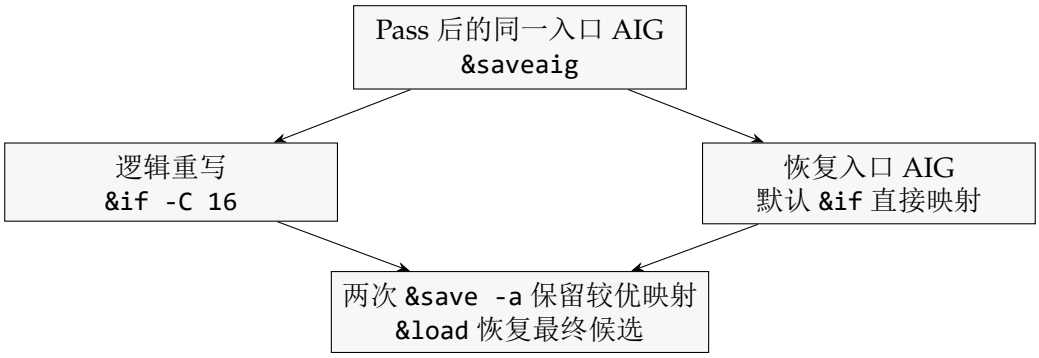


图 2.16 ABC9 两个映射候选的生成与选择

2.7.3 候选选择与深度评价

`&save -a` 调用配套 ABC 的面积优先比较：优先保留 LUT 数较少的候选；LUT 数相同时，保留 LUT 深度较小的候选。若二者均相同则保留已有结果。`&load` 恢复所保存

的映射，`&ps -l` 打印 LUT 分布统计，之后 Yosys 读回结果并继续 Cyclone IV 单元映射。

候选生成考虑延迟模型，选择以面积优先。所有案例统一生成两路候选，由 ABC 保存与恢复；最终并列统计 LUT 数与深度。

第三部分 正确性验证与性能评估

3.1 测试配置与评价方法

评估按统一案例集、功能正确性、面积贡献、运行代价及额外适用边界展开。默认案例集为 `project/cases/` 顶层的九个输入；`P2/` 和 `pressure/` 为另外指定目录运行的专项及压力输入，不与默认九例混合计算平均优化率。

3.1.1 软件配置与对照原则

当前交付程序为 Windows x64 原生 Yosys。基线和优化版均以正式 v0.69 提交 `9f75ca1f9834` 为基础，配套 ABC 子模块均为 `a51bf4a7bb38`，采用相同 MSYS2 UCRT64、GCC/G++ 16.2.0 及 Release 构建配置。两版共用 `synthesis.ys`，在独立工作目录中读取同一冻结输入并保存结果。

等价证明使用 OSS CAD Suite 中的 EQY、SBY、证明用 Yosys 和 ABC，与被测综合程序分开 [6]。各次运行的工具版本及配置记录在运行清单中。编译方法和工程组织见第四部分。

3.1.2 指标定义与结果分组

面积采用最终 Cyclone IV 映射后的 `cycloneiv_lcell_comb` 与独立 NOT 数量之和，DFF 单独统计。设第 i 例的组合逻辑数量为 N_i，优化率为

$$R_i = \frac{N_{\mathrm{base},i} - N_{\mathrm{opt},i}}{N_{\mathrm{base},i}} \times 100\%, \qquad \overline{R} = \frac{1}{m}\sum_{i=1}^{m} R_i.$$

逐例算术平均用于案例评价，合计削减率用于反映总资源变化。LUT 深度由映射网络中串联 LUT 的最大层数统计，用于观察映射网络的结构深度变化。

效率使用完整综合的墙钟时间及 Yosys 主进程峰值工作集，定义详见第 3.4 节。超时、UNKNOWN 或工具执行错误不计作等价通过。专项实验分别列明启用规则及 ABC9 命令流。

3.2 功能等价性验证

3.2.1 比较对象与验证检查点

验证同时检查原始 RTL 的设计行为和两版综合结果的实现行为。每个案例固定为同一份输入文件，再由基线版、优化版分别综合。证明检查点位于 ABC9 完成之后、最

终 Cyclone IV `map_cells` 之前，保存为 `formal.rtlil`，覆盖 Pass 改写、后续逻辑优化及 LUT 映射；最终资源数量在器件单元映射完成后统计。

表 3.1 三组整设计等价比较及其作用

比较对象	检查目的
原始 RTL 与基线网表	检查基线综合结果与 RTL 行为的对应关系。
原始 RTL 与优化网表	检查完整优化流程是否保持 RTL 规定的行为。
基线网表与优化网表	直接比较两版映射结果的初始输出及后续行为。

三组比较同时检查初态及后续输出序列。例如，D 端常量化可能引起寄存器删除，影响器件上电状态的对应关系。P3 保留必要的信号连接，并通过两版网表比较核对该行为。

3.2.2 统一建模与整设计证明机制

证明前先将两侧电路转换为可统一比较的逻辑和状态模型。RTL 一侧通过 `memory_map` 展开 ROM 等存储结构；映射网表一侧按照真值表展开 `$lut`，并用 `dfflegs_unmap.v` 将支持的器件触发器配置还原为普通触发器描述。模型保留 `power_up` 对应的确定初态，未支持的控制配置由检查报错。两侧均展平层次，并使用 `clk2fflogic` 表示时钟边沿和状态更新，使时钟、使能及异步控制进入统一证明模型。

EQY 按官方流程构造比较电路 [7]，让两侧接收相同输入序列，并按各自初态和状态转移关系逐位比较输出；未定义值按 EQY 语义处理。

工程采用整设计分区。局部分区会将切分边界上的信号作为自由输入，原电路中跨越这些边界的状态关联可能随之丢失。配置先用 `rename -hide` 隐藏内部匹配信号，再将全部顶层输出收集到一个`.whole` 分区，使输出比较保留完整设计的组合连接与状态关系。关键配置为：

代码 3.1 compare.eqy 中的整设计分区与证明策略

```
[collect *]
group *
bind *

[partition *]
name whole *

[strategy pdr]
use sby
engine abc pdr
timeout 400
```

EQY 通过 SBY 调用 ABC PDR 引擎，以归纳不变式证明所有可达状态下的输出条件，或给出反例 [8]。证明用 ABC 与综合阶段的 ABC9 分别承担形式验证和 LUT 映射。

3.2.3 通过条件与验证结果

`bench.py` 中的 `run_eqy()` 分别生成三组配置并保存日志。接受单项证明时，既检查工具结论，也读取分区 JSON 核对实际证明范围：

1. EQY 返回码为 0，最终结论为 `DONE (PASS, rc=0)`，并含成功证明设计等价的记录。
2. 仅有一个以`.whole` 结尾的整设计分区。
3. 分区输出与顶层全部输出逐位一致。
4. 分区输入均为合法顶层输入，且没有额外跨分区信号。

这些检查将 PASS 结论与完整输出覆盖对应起来。每组策略超时上限为 400 秒；有效反例、超时、UNKNOWN 和工具错误分别记录，三组均通过后才将案例记为验证通过。

九例均通过三组整设计比较，共 27 项证明。

3.3 面积优化效果与贡献分析

3.3.1 九个案例的整体优化结果

九例的最终资源、DFF、LUT 深度和等价性结果见第 1.3 节表 1.2。组合逻辑单元合计由 2427 降至 1985，逐例优化率算术平均为 24.21%；DFF 合计由 323 降至 320。LUT 深度五例不变、两例降低、两例增加。下文通过命令流对照和规则消融说明收益来源。

3.3.2 Pass 与 ABC9 命令流的贡献对照

三种配置使用同一份输入及综合脚本：A 为无 Pass 与原版 ABC9 命令流，B 为完整 Pass 与原版 ABC9 命令流，C 为完整 Pass 与优化版 ABC9 命令流。A、B 比较结构优化的贡献，B、C 比较配套命令流在同一 Pass 基础上的增量。

表 3.2 九例在三种配置下的资源与 LUT 深度

案例	A：无 Pass ＋原版命令流			B：完整 Pass ＋原版命令流			C：完整 Pass ＋优化版命令流		
	LUT ＋ NOT	DFF	LUT 深度	LUT ＋ NOT	DFF	LUT 深度	LUT ＋ NOT	DFF	LUT 深度
test1	288	222	4	184	222	4	184	222	4
test2	25	0	2	15	0	2	15	0	2
test3	13	17	3	8	14	3	8	14	3
test4	36	10	3	29	10	3	26	10	2
balanced	426	12	10	402	12	10	366	12	9
pipeline	450	24	4	391	24	5	348	24	6
crossbar	408	10	6	363	10	7	361	10	7
feedback	368	16	7	295	16	7	299	16	7
nested	413	12	10	409	12	10	378	12	10
合计	2427	323	—	2096	320	—	1985	320	—

Pass 使组合逻辑单元合计减少 331 个，优化版 ABC9 命令流再减少 111 个。test1、test2、test3 在 B、C 下相同，资源节省主要来自 Pass；balanced、pipeline、nested 分别进一步减少 36、43、31 个单元，test4 减少 3 个且 LUT 深度由 3 降为 2。feedback 的单元数由 295 增至 299，深度仍为 7；pipeline 的单元数由 391 降至 348，深度由 5 增至 6。DFF 数量在 B、C 间保持一致。

面积收益与 LUT 深度变化

九例中，test1、test2、test3、feedback、nested 的最大 LUT 层数保持不变，test4 和 balanced 分别降低一层。pipeline 的组合逻辑单元由 450 减至 348，削减率为 22.67%，层数由 4 增至 6；crossbar 由 408 减至 361，削减率为 11.52%，层数由 6 增至 7。

三组配置进一步给出了变化来源：pipeline 在加入 Pass 后由 4 层变为 5 层，采用优化版 ABC9 命令流后再变为 6 层；crossbar 在 Pass 阶段由 6 层变为 7 层，后续候选选择保持 7 层。面积优先选择允许较少 LUT 的候选具有更深的逻辑层级，这两例体现了资源节省与结构深度之间的取舍。

本方案以组合逻辑面积为优化目标，并列统计最大 LUT 层数。对于存储、选择及并行数据通路，可结合资源预算和关键路径要求选择规则组合；对路径延迟约束较紧的设计，应进一步结合器件时序模型和布局布线结果评估。LUT 层数描述串联逻辑级数，实际延迟还受单元引脚、扇出和布线影响。

P2 输入变体与映射适配

P2 专项用于观察同一优先选择链在默认输入变化、中间结果复用和位宽扩展时的映射表现。下表覆盖八种输入变体，比较无 Pass 与仅启用 P2，两侧均使用原版 ABC9 命令流。

表 3.3 P2 补充案例在原版 ABC9 命令流下的测试结果

输入结构	位宽	无优化 LUT	仅新 P2 LUT	面积变化	LUT 深度
默认零	4	20	16	−20.00%	2 → 2
默认零	16	80	64	−20.00%	2 → 2
默认独立信号	4	20	16	−20.00%	2 → 2
默认独立信号	16	80	64	−20.00%	2 → 2
默认 `Z ^ Q`	8	48	40	−16.67%	3 → 2
默认 `A & Z`	8	40	32	−20.00%	2 → 2
默认独立信号，下半段另有输出	8	40	41	+2.50%	3 → 2
默认来自另一级 MUX，共五级链	8	48	33	−31.25%	3 → 3

默认输入为常量、独立信号或组合函数时，P2 均可重组原优先链；不同位宽下保留相同的变换形式。下半段结果被额外输出使用时，两个输出共同约束映射，LUT 数由 40

增至 41，深度由 3 降至 2。
两个默认零变体的命令流对照见表 3.4。三种配置均无独立 NOT 和 DFF，LUT 深度均为 2；表内列出 LUT 数量。

表 3.4 P2 默认零结构与 ABC9 命令流的适配

案例	无 Pass + 原版命令流	仅 P2 + 原版命令流	仅 P2 + 优化版命令流
zero4	20	16	12
zero16	80	64	48

P2 将四级优先链重组为并行数据选择，原版命令流映射后平均每位需要四个 LUT，优化版命令流进一步得到每位三个 LUT。正式 test2 则在两种命令流下均为 15 个 LUT。完整九例对照与这组输入变体分别展示总体贡献和结构适配机制。

3.3.3 关键规则的消融与收益来源

消融实验固定使用原版 ABC9 命令流，在相同 Pass 接入位置逐步加入规则内部处理。临时实验版本仅控制相应步骤的开关。各配置统计组合逻辑单元、DFF 与 LUT 深度，并执行三组整设计等价性验证。九例命令流对照和规则消融共 26 组配置比较、78 项整设计证明，全部通过；两个 P2 默认零变体的命令流对照另完成 12 项证明，全部通过。

P1：完全交换共享与部分交换

test1 的后七组具有完整交换关系，第一组需要局部 MUX 处理。先仅共享后七组，将第一组保留在两个原 PMUX 中；再将第一组通过局部 MUX 纳入共享网络，比较共享范围变化的作用。

表 3.5 P1 两种共享范围的消融对照

配置	LUT + NOT	DFF	LUT 深度
无 Pass	288	222	4
仅共享完全交换组	247	222	4
完整 P1（含部分交换）	184	222	4

仅共享完全交换组节省 41 个组合逻辑单元；纳入部分交换后再减少 63 个。第一组增加的局部 MUX 使全部八组能够共用同一数据选择网络，省去了两个输出各自处理残余通道的选择逻辑。三种配置的 DFF 数量和 LUT 深度相同。

P3：数据端化简、使能复用与比较共享

P3 按实际执行顺序累计启用三个步骤，观察每一步对最终映射的作用：

表 3.6　P3 三个处理步骤的累积消融

配置	LUT + NOT	DFF	LUT 深度
无 Pass	13	17	3
数据端化简	12	14	3
数据端化简＋使能复用	10	14	3
完整 P3（再加比较共享）	8	14	3

数据端化简把受使能约束的写入选择替换为已有信号位，减少一个组合逻辑单元，并使三位寄存器满足后续合并条件。使能复用与比较共享分别再减少两个单元。三个步骤的累积结果保持 3 层 LUT 深度，分别利用了写入条件、已有控制信号和公共比较位。

P4：基础分解、编码候选与联合选择

P4 的基础方案按地址位展开；第二种配置增加编码分解候选，但每个输出独立择优；完整配置进一步按所有输出的共享节点联合选择。三种方案均保留正式规则的采用门槛。

表 3.7　P4 候选生成与联合选择的消融对照

配置	LUT + NOT	DFF	LUT 深度
无 Pass	36	10	3
基础地址位展开	34	10	3
增加编码候选、逐输出择优	34	10	3
完整 P4：跨输出联合选择	29	10	3

基础展开方案的网络成本为 26 个 LUT 节点；逐输出择优得到 23 个，节省量为 3，未达到本例 4 个节点的采用门槛，因而仍使用基础方案。联合选择将共享网络成本降至 21 个节点，满足门槛并被采用，最终映射比基础方案再减少 5 个组合逻辑单元。编码候选与跨输出共享共同作用，DFF 和 LUT 深度保持不变。完整方案再配合优化版 ABC9 命令流后得到 26 个组合逻辑单元、2 层 LUT，见表 3.2。

3.4　运行时间与峰值内存评估

3.4.1　统一测量范围与比较条件

效率比较采用两版相同的输入、综合脚本、测量工具和运行顺序。`bench.py` 先冻结输入文件，再让两版在独立工作目录中执行同一份 `synthesis.ys`。测量覆盖从启动被测 Yosys 到退出的完整综合过程，包括读入 RTL、Pass、ABC 运行及等待、后续器件映射、检查和结果输出。案例准备、测量器初始化、EQY 证明及报告生成在该范围之外。

时间采用 PowerShell `Measure-Command` 返回的墙钟时间；内存采用同次运行中 Yosys 主进程的峰值工作集。两版均按此口径采集，并保存对应的原始测量记录。

3.4.2 测量实现与数据完整性

`measure_yosys.ps1` 调用预编译的 C# 测量类 `PmuxWindowsMeasurement`。对象构造阶段完成路径、进程参数及日志文件准备；计时阶段启动 Yosys，并等待其退出。核心片段如下：

代码 3.2 测量脚本中的时间与内存采集

```
$elapsed = Measure-Command {
    $measurement.Start()
    $measurement.Wait()
}
$peakBytes = $measurement.ReadPeakWorkingSet()
$measurement.FinishLogs()
$exitCode = $measurement.ExitCode
```

计时覆盖进程启动至退出，包括 Yosys 等待 ABC 的时间；日志随运行异步转存，退出后的收尾在计时区间之外。

峰值内存通过进程句柄调用 Windows API `GetProcessMemoryInfo` 读取 `PeakWorkingSetSize`，在退出后、释放句柄前取得操作系统记录的生命周期峰值，并除以 2^{20} 转为 MiB。

每次测量将返回码、墙钟时间、峰值内存及统计来源保存为 `measurement.json`。`bench.py` 只有在综合正常退出、测量有效且时间与内存均为正值时接受样本。结果同时保留综合日志，便于从汇总指标追溯到具体运行。

3.4.3 预热复核与开销计算

短时墙钟测量受文件缓存、进程启动、系统调度和后台负载影响。本工程先完成整批预热，再开始正式比较：冻结全部所选输入，各例用基线完整综合一次；随后逐例执行基线和优化版测量。单例运行也预热一次，预热数据单独打印并保留。

当前 `apply_measurement_review()` 在基线首测小于 10 秒、且时间开销超过 5% 时追加五组基线、优化版配对复核。复核后，分别计算两版五次时间均值，再计算开销；首测保留为记录。设两版时间分别为 T_b, T_o，内存为 M_b, M_o，则

$$\Delta T = (T_o/T_b - 1) \times 100\%, \qquad \Delta M = (M_o/M_b - 1) \times 100\%.$$

复核同时检查每次资源统计与首测一致。时间均值按 $T_b = \frac{1}{5}\sum_i T_{b,i}$、$T_o = \frac{1}{5}\sum_i T_{o,i}$ 计算。内存使用本工程的配对取值约定：从首测及五组复核中，选择 M_o/M_b 最大的一组，保留该组两版峰值。样本、选择依据及最终数值一并保存。

多轮评估汇总各轮开销均值、范围及达标次数，原始样本和资源一致性检查一并保

留。

3.4.4 运行结果与波动分析

九例的时间与主进程峰值内存汇总采用第 1.3 节表 1.3。十轮中时间开销最高为 test3 的 3.23%，九例时间均达标；八例峰值内存开销始终低于 5%，crossbar 的内存开销为 4.23%～6.42%，两项同时达标 5 轮。

3.5 补充压力测试

3.5.1 输入结构与规模

赛事方提供的案例中，Pass 入口规模最大的是 `pmux_case02_pipeline`，含 152 个单元。为补充千节点规模测试，自行构造两个压力案例，输入位于 `project/cases/pressure/`。普通数据通路案例包含 128 组算术、移位及寄存逻辑；混合结构案例包含 96 组比较、选择、查表及寄存逻辑。两个案例均使用彼此独立的输入和控制信号扩展规模，并保留全部输出。

表 3.8 补充压力案例的入口规模与映射结果

案例	Pass 入口		组合逻辑单元		DFF	LUT 深度
	cell	PMUX	原版	优化版	原版 / 优化版	原版 / 优化版
普通数据通路	1282	0	12127	12127	1024 / 1024	8 / 8
混合结构	1986	64	16181	15550	1536 / 1536	10 / 10

普通数据通路用于检查无目标结构时的规模开销，混合结构用于检查比较、选择及查表网络共同存在时的搜索与重组。混合结构减少 631 个组合逻辑单元，两个案例的 DFF 数量及 LUT 深度均保持不变。

3.5.2 运行代价与整设计验证

两例采用与九例相同的综合程序、综合脚本和指标口径。整批预热后，逐例测量原版与优化版；本表为一次配对运行的实测记录。

表 3.9 补充压力案例的墙钟时间与主进程峰值内存

案例	墙钟时间（s）			峰值内存占用（MiB）		
	原版	优化版	增量	原版	优化版	增量
普通数据通路	14.619	15.834	+8.31%	243.531	243.480	−0.02%
混合结构	21.864	19.740	−9.72%	299.492	304.391	+1.64%

普通数据通路没有面积收益，本轮时间增量为 8.31%；混合结构在面积减少的同时，墙钟时间下降 9.72%。两例主进程峰值内存增量均低于 5%。

两个压力案例的原版与优化版整设计比较均通过。普通数据通路的三组比较全部通过；混合结构的 RTL 与原版比较通过，RTL 与优化版比较在 900 秒策略时限下超时。

3.5.3 压力证明的工具配置

压力案例的综合均正常完成，表 3.9 中的主进程峰值内存增量分别为 −0.02% 和 +1.64%。验证阶段，Windows 版 OSS CAD Suite 的证明用 Yosys 在 EQY 分区时触发 `0xC00000FD`，对应验证工具可执行文件的栈预留限制。实验在独立副本上将栈预留由 2 MiB 调为 128 MiB，消除该栈溢出后继续证明：普通数据通路的三组比较全部通过；混合结构的 RTL 与基线网表、基线与优化网表两组比较通过。混合结构的 RTL 与优化网表比较在 900 秒策略时限下仍超时，保留为未完成证明。栈预留调整用于解决验证工具的执行限制，综合程序与优化规则保持不变。九个正式案例使用原始验证工具配置。

第四部分 工程组织与复现

工程以 Windows 原生程序交付，包含基线版、优化版、运行依赖、优化源码和九例测试。工程根目录的 `README.md` 为统一操作指南，给出验证工具准备、环境配置、源码下载、文件放置、双版本编译及测试命令。

4.1 交付内容与文件组织

两版程序目录的 `build/` 保存 Yosys、配套 ABC、共享库及器件数据，`runtime/` 保存运行依赖。优化源码、测试输入和自动化脚本分别位于 `project/src/`、`project/cases/` 和 `project/runner/`；源码版本及文件对应关系见 README。

4.2 运行与重新构建

受提交材料体积限制，验证套件需单独准备。按 README 的“快速运行”下载 Windows x64 OSS CAD Suite 20260830，将 `oss-cad-suite/` 放在工程根目录，与 `run.cmd` 同级；已有安装也可按指南配置绝对路径。随后在工程根目录打开 PowerShell：

```
.\run.cmd --check          # 检查环境
.\run.cmd                  # 运行九例测试
.\run.cmd --case test3     # 运行单例
```

入口自动完成预热、两版综合、指标采集、必要复核及三组等价证明。结果打印于控制台，并保存至 `project/reports/` 和 `project/runs/`；专项与压力案例的运行命令同样列于 README。

重新构建依次执行 README 的“构建环境”和“从源码重新编译”：安装 MSYS2 UCRT64 工具，下载 base 并复制为 opt，放入优化文件，分别配置、编译和检查。

参考文献

[1] YosysHQ. Yosys documentation：the RTL intermediate language（RTLIL）[EB/OL]. [2026-11-01]. https://yosyshq.readthedocs.io/projects/yosys/en/stable/yosys_internals/formats/rtlil_rep.html.

[2] YosysHQ. Yosys documentation：multiplexers[EB/OL]. [2026-11-01]. https://yosyshq.readthedocs.io/projects/yosys/en/stable/cell/word_mux.html.

[3] Altera Corporation. Cyclone IV device handbook，volume 1，chapter 3：memory blocks in cyclone IV devices[EB/OL]. [2026-11-01]. https://www.intel.com/content/www/us/en/content-details/654744/cyclone-iv-device-handbook-volume-1-chapter-3-memory-blocks.html.

[4] YosysHQ. Yosys documentation：the ABC toolbox[EB/OL]. [2026-11-01]. https://yosyshq.readthedocs.io/projects/yosys/en/stable/using_yosys/synthesis/abc.html.

[5] Berkeley ABC. ABC：system for sequential logic synthesis and formal verification [EB/OL]. [2026-11-01]. https://github.com/berkeley-abc/abc.

[6] YosysHQ. EQY documentation：installing[EB/OL]. [2026-11-01]. https://yosyshq.readthedocs.io/projects/eqy/en/latest/install.html.

[7] YosysHQ. EQY documentation：getting started[EB/OL]. [2026-11-01]. https://yosyshq.readthedocs.io/projects/eqy/en/latest/quickstart.html.

[8] Brayton R，Mishchenko A. ABC：an academic industrial-strength verification tool [C/OL]//Lecture Notes in Computer Science: v.6174 Computer aided verification. Springer，2010：24-40. DOI:10.1007/978-3-642-14295-6_5.